

第二章 密码学

顾辰

guchen@hfut.edu.cn



2.1 密码学的基本概念

□ 算法(Algorithm): 解决复杂问题的步骤

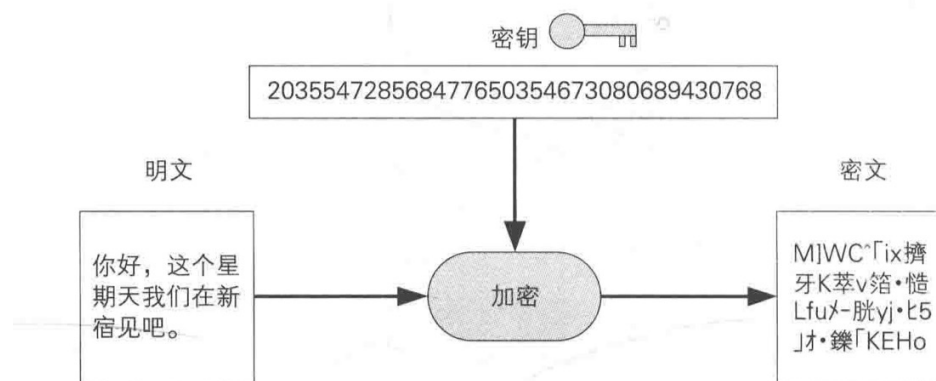
➤ 加密算法

➤ 解密算法

□ 明文(Plaintext): 须要加密的消息或数据，是加密算法的输入。

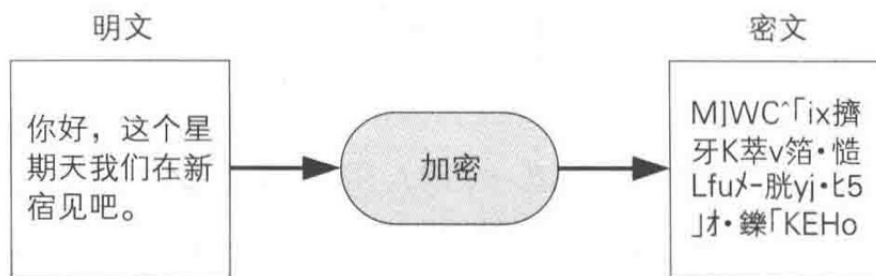
□ 密文(Ciphertext): 加密之后的数据，是加密算法的输出。

□ 密钥(Key): 加密或解密的参数。不同的密钥，会有不同的输出结果。

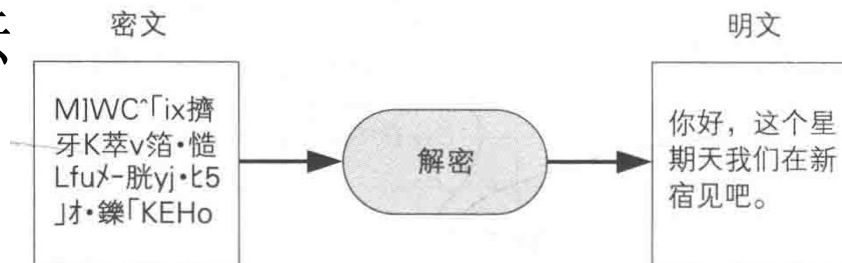


2.1 密码学的基本概念

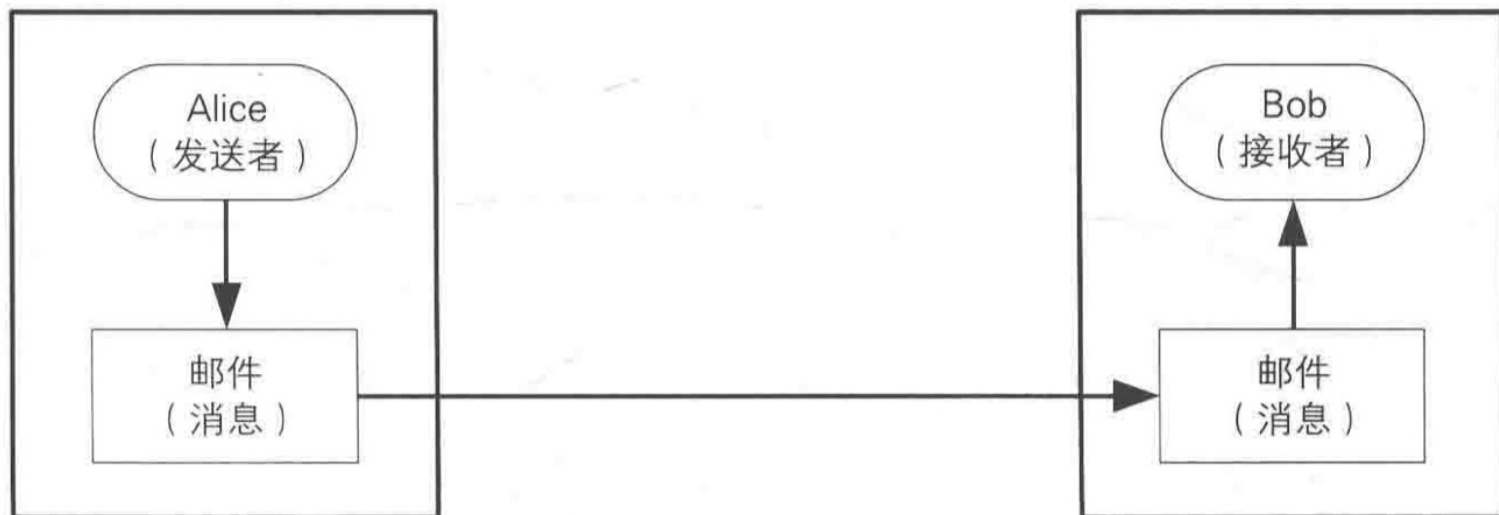
- 加密(Encryption): 将明文变成密文的过程。 $C=E_K(P)$, K 是密钥, P 是明文, C 是密文, E 是加密算法。



- 解密(Decryption): 将密文恢复成明文的过程。 $P=D_K(C)$, D 是解密算法

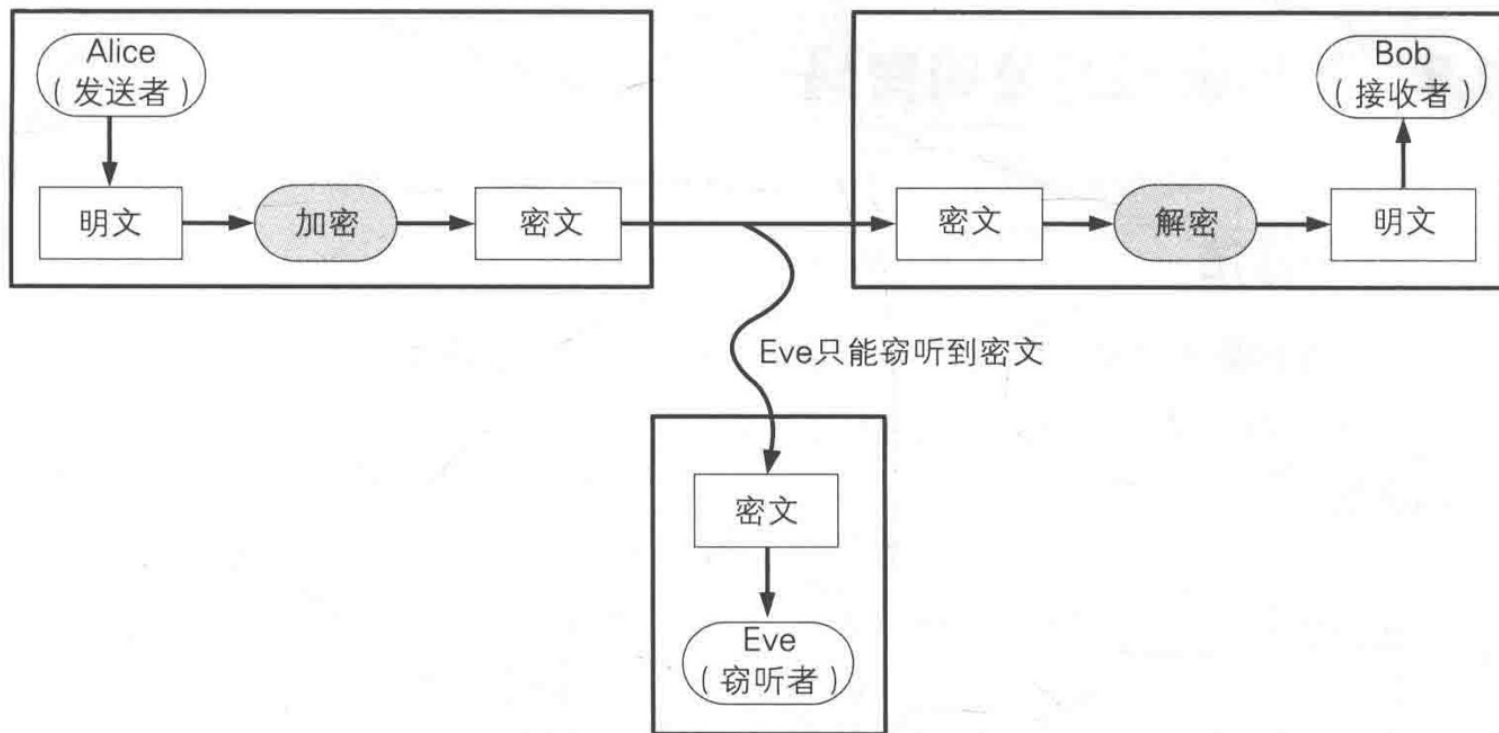


2.1 密码学的基本概念



□ Alice向Bob发送数据

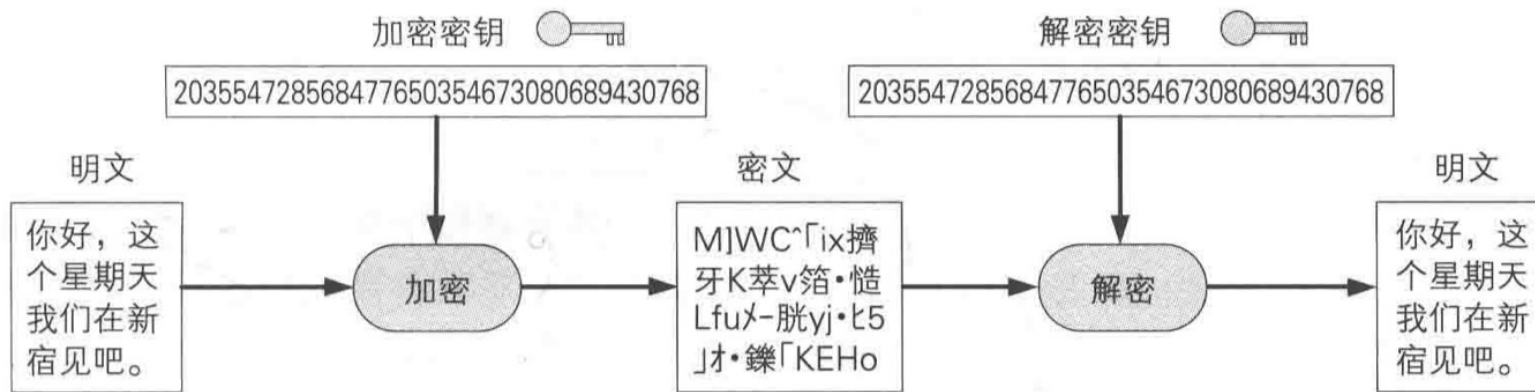
2.1 密码学的基本概念



□ 将消息加密发出后，窃听者只能得到密文

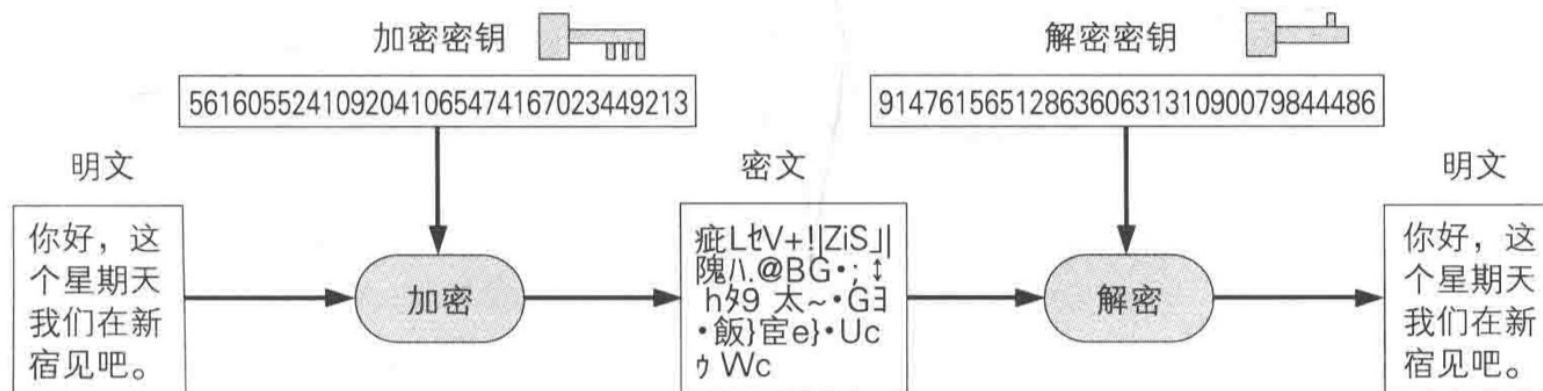
2.1 对称密码

对称密码中，加密用的密钥和
解密用的密钥是相同的



- 对称密码是指加密和解密时使用同一把密钥的方式。

2.1 非对称密码（公钥密码）

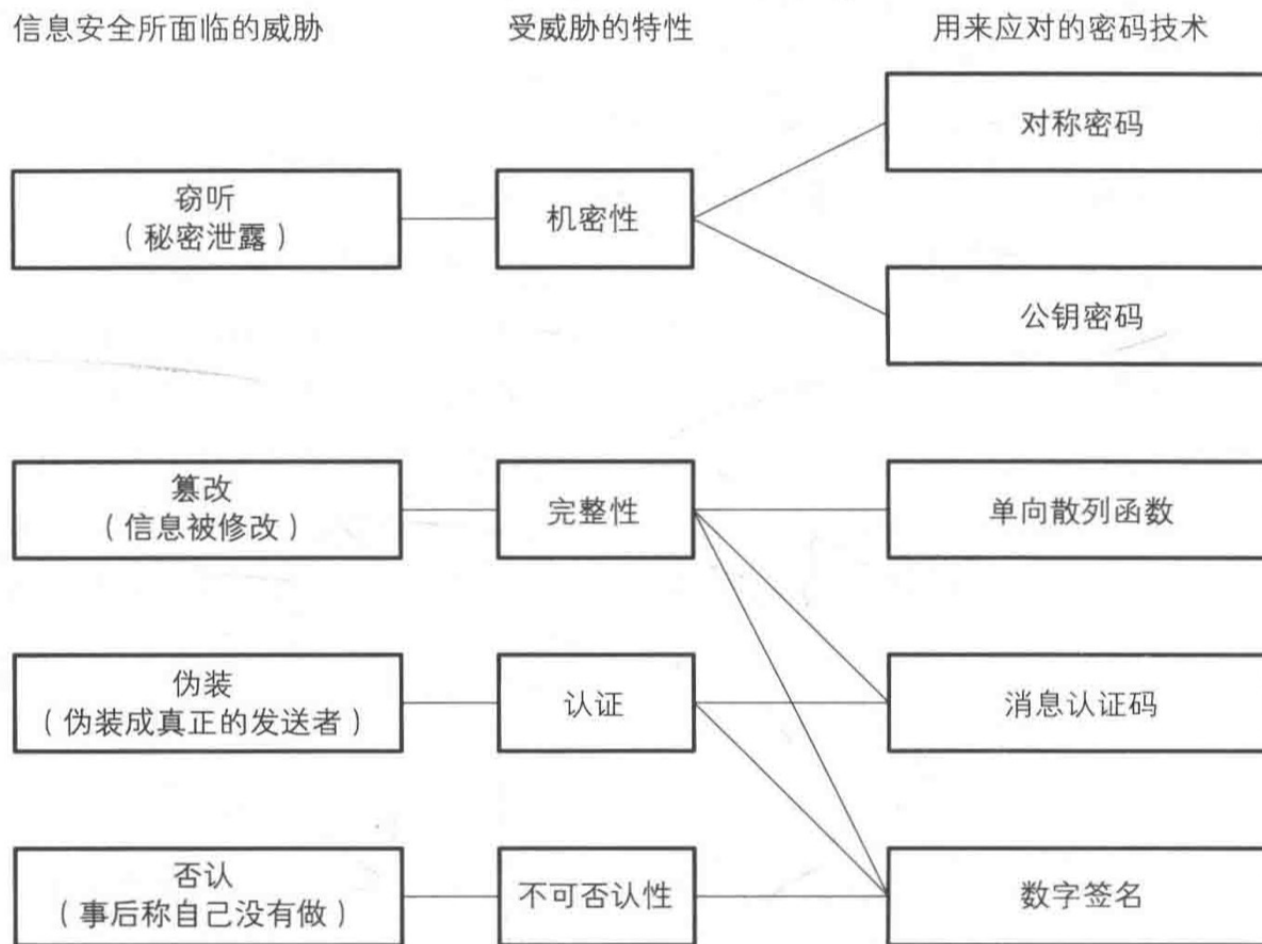


- 非对称密码是指加密和解密时使用不同密钥的方式。

2.1 其他密码技术

- 单项散列函数（one-way hash function）：检测数据是否被篡改过，保证数据的**完整性**。
- 消息认证码（message authentication code）：不仅能检测消息是否被篡改，而且能确认消息是否来自于所期待的通信对象，提供**认证**机制。
- 数字签名（digital signature）：保证完整性、提供认证且防止**否认**。
- 伪随机数生成器：模拟产生随机数的算法，承担**密钥生成**的重要职责。

2.1 密码学家的工具箱



2.1 信息隐藏（隐写）

我们先准备一段话，
很容易看懂的就可以，
喜闻乐见的当然更好。
欢迎你尝试将另一句话嵌在这段话中，
你会发现这其实就是一种隐写术。

- 现代的信息隐藏常将秘密信息藏于多媒体文件（如视频、图片）中。
- 隐写与水印

2.1 密码与信息安全常识

- ❑ 不要使用保密的密码算法
- ❑ 开发高强度的密码算法是非常困难的
- ❑ 任何密码总要一天都会被破解
- ❑ 密码只是信息安全的一部分

2.2 历史上的密码

莫尔斯密码

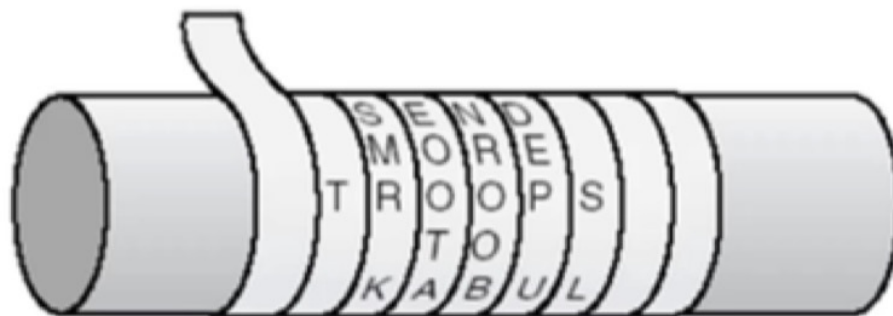
- 由点di (.)、划da (-) 两种符号按以下原则组成：
- 其他定义：划一般是三个点的长度；点划之间的间隔是一个点的长度；字符之间的间隔是三个点的长度；单词之间的间隔是七个点的长度。

字符	电码符号	字符	电码符号	字符	电码符号	字符	电码符号
A	. -	B	- . . .	C	- . - .	D	- . .
E	.	F	. . - .	G	- - .	H	
I	. .	J	. - - -	K	- . -	L	. - . .
M	- -	N	- .	O	- - -	P	. - - .
Q	- - . -	R	. - .	S	. . .	T	-
U	. . -	V	. . . -	W	. - -	X	- . . -
Y	- . - -	Z	- - . .				

- I LOVE YOU就是 . . . - . . - - - . . . - . - - - - . -

2.2 历史上的密码

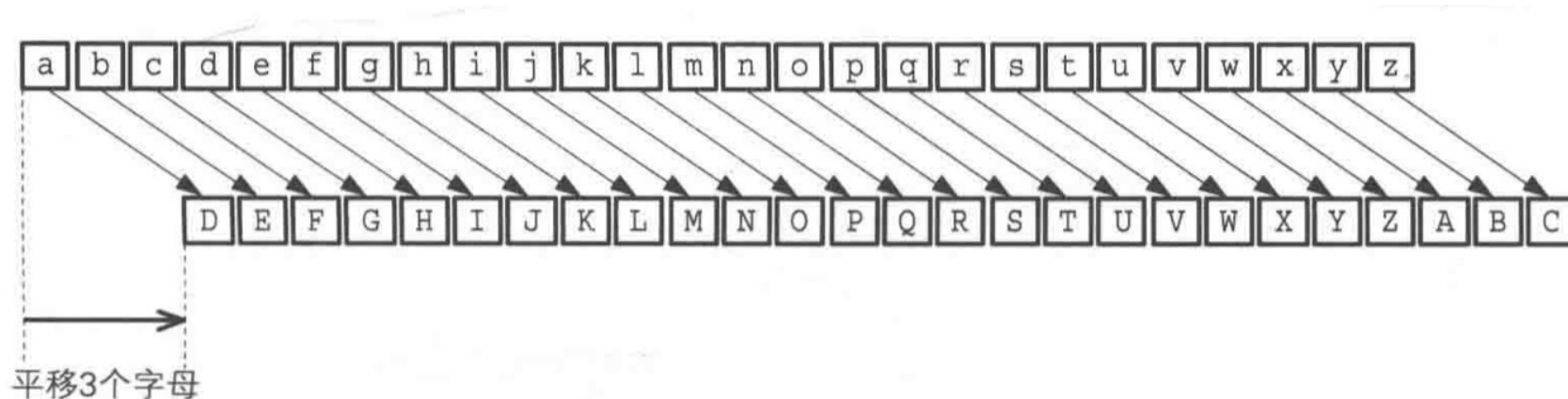
500 B. C.，古斯巴达人使用的“天书”



2.2 历史上的密码

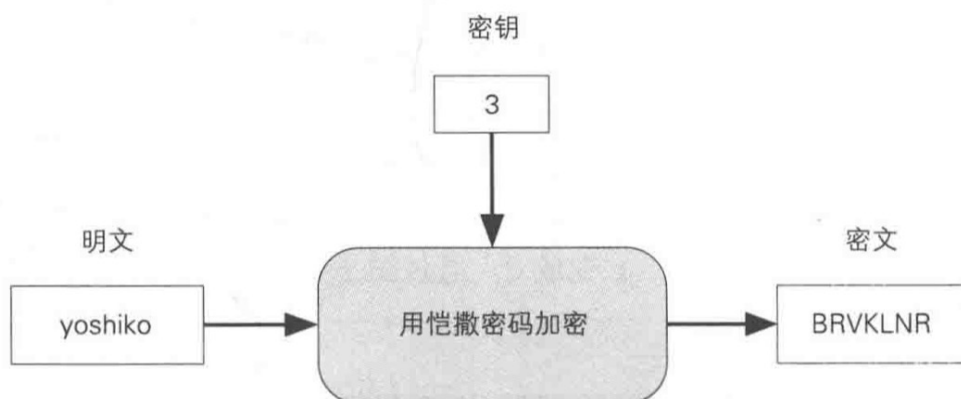
□ 凯撒密码

- 恺撒密码（Caesar cipher）是一种相传尤利乌斯·恺撒曾使用过的密码。恺撒于公元前 100 年左右诞生于古罗马，是一位著名的军事统帅。
- 恺撒密码是通过将明文中所使用的字母表按照一定的字数“平移”来进行加密的。

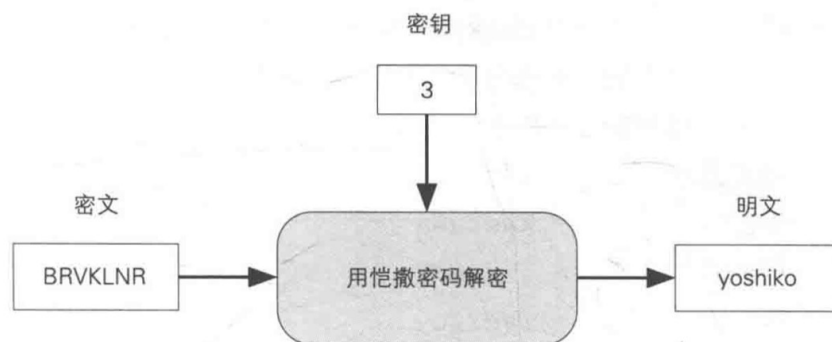


2.2 历史上的密码

凯撒密码加密



凯撒密码解密



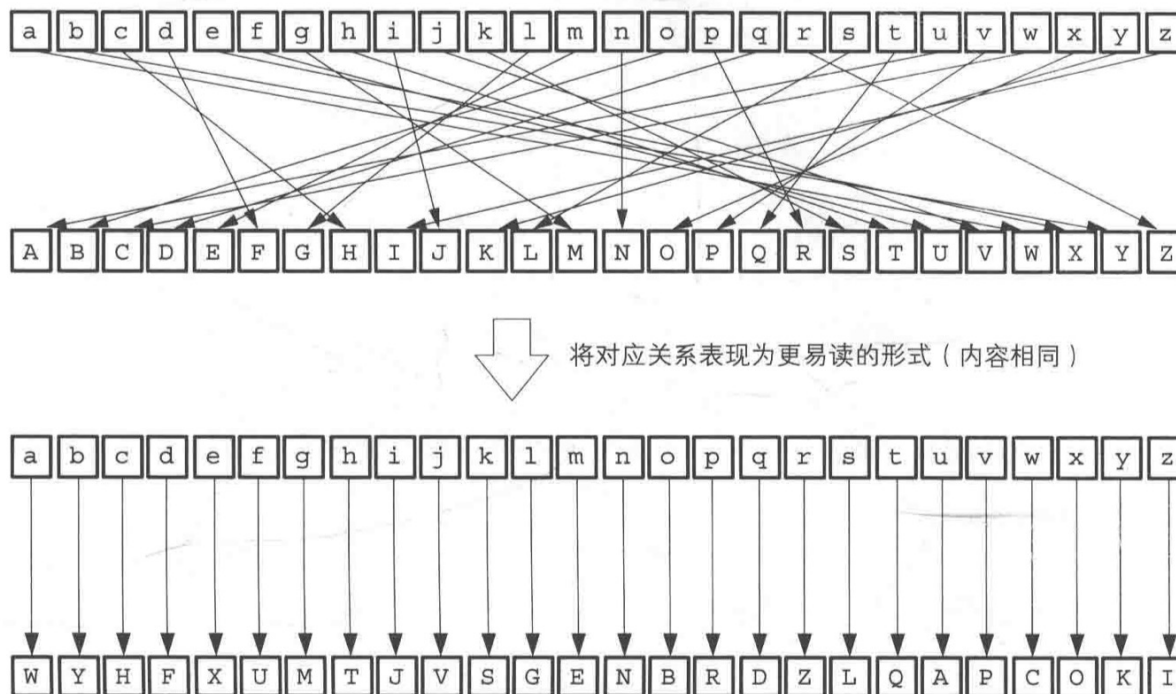
2.2 历史上的密码

□ 凯撒密码的破译？

2.2 历史上的密码

□ 简单替换密码

- 将明文中所使用的字母替换成另一套字母表的密码



2.2 历史上的密码

- 简单替换密码的破译？
 - 庞大的密钥空间（ $26!$ ）
 - 难以用暴力破解来破译
 - 思考：如何破解？



2.3 从文字到比特序列密码

□ 编码

□ 将字母数字等映射为比特序列 (ASCII)

□ 异或 (XOR)

$$0 \text{ XOR } 0 = 0$$

(0 与 0 的 XOR 结果为 0)

$$0 \text{ XOR } 1 = 1$$

(0 与 1 的 XOR 结果为 1)

$$1 \text{ XOR } 0 = 1$$

(1 与 0 的 XOR 结果为 1)

$$1 \text{ XOR } 1 = 0$$

(1 与 1 的 XOR 结果为 0)

m → 01101101

i → 01101001

d → 01100100

n → 01101110

i → 01101001

g → 01100111

h → 01101000

t → 01110100

□ 比特序列的异或

$$\begin{array}{r} 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ \dots \ A \\ \oplus \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ \dots \ B \\ \hline 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ \dots \ A \oplus B \end{array}$$

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ \dots \ A \oplus B \\ \oplus \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ \dots \ B \\ \hline 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ \dots \ A \end{array} \quad (\text{变回了 } A)$$

2.3 一次性密码本

- 将明文与一串随机的比特序列进行XOR运算
- 加密

01101101	01101001	01100100	01101110	01101001	01100111	01101000	01110100	明文“midnight”
⊕ 01101011	11111010	01001000	11011000	01100101	11010101	10101111	00011100	密钥
00000110	10010011	00101100	10110110	00001100	10110010	11000111	01101000	密文

- 解密

00000110	10010011	00101100	10110110	00001100	10110010	11000111	01101000	密文
⊕ 01101011	11111010	01001000	11011000	01100101	11010101	10101111	00011100	密钥
01101101	01101001	01100100	01101110	01101001	01100111	01101000	01110100	解密后得到明文 midnight

2.3 一次性密码本

- 一次性密码本是由维纳(G.S. Vernam)于1917年提出，因此又称维纳密码。
- 一次性密码本是无法破解的
 - 无法判断它是否是正确的明文
- 一次性密码本无法破解这一特性由香农(C.E.Shannon)于1949年通过数学方法加以证明的。
- 一次性密码本是**无条件安全的、在理论上是无法破译的**。

思考：为什么一次性密码本没有被使用？

2.3 一次性密码本

为什么一次性密码本没有被使用？

- 密钥的配送

- 密钥压缩是否可行？如果不可行，会造成什么问题？

- 密钥的保存

- 密钥的重用（密钥不能重复）

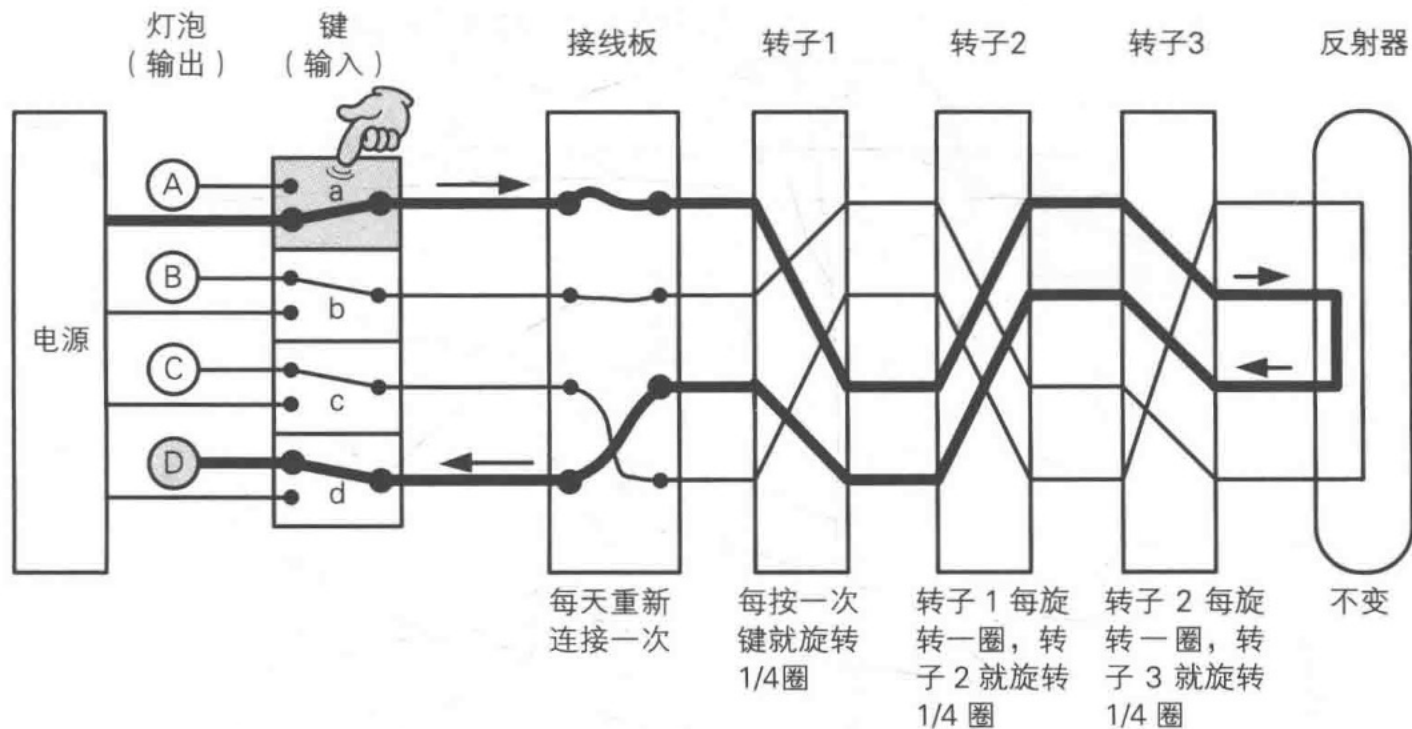
- 密钥的同步（密钥错位）

- 密钥的生成（随机性）

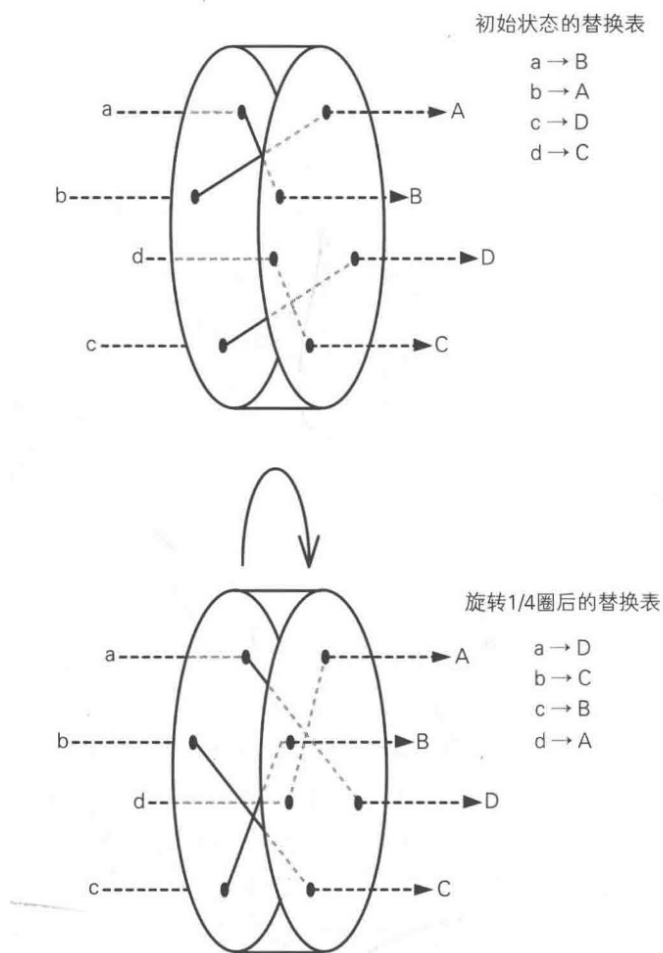
2.3 Enigma

- Enigma 是由德国工程师阿瑟·谢尔比乌斯（[Arthur Scherbius](#)）于 20 世纪初发明的一种可进行加密与解密操作的机械密码设备。
- 通过引入可旋转转子结构与电路连接机制，构建出一种能够生成高度复杂替换密码的加密系统，其安全强度远超当时人工手工加密方式所能达到的水平。
- Enigma 在问世初期主要应用于商业通信领域，随后在纳粹德国时期被德国国防军采用，并在原有基础上进行改进和强化，广泛应用于军事通信加密之中。

2.3 Enigma



2.3 Enigma



战争密码（上集）如何复刻一台恩格玛机

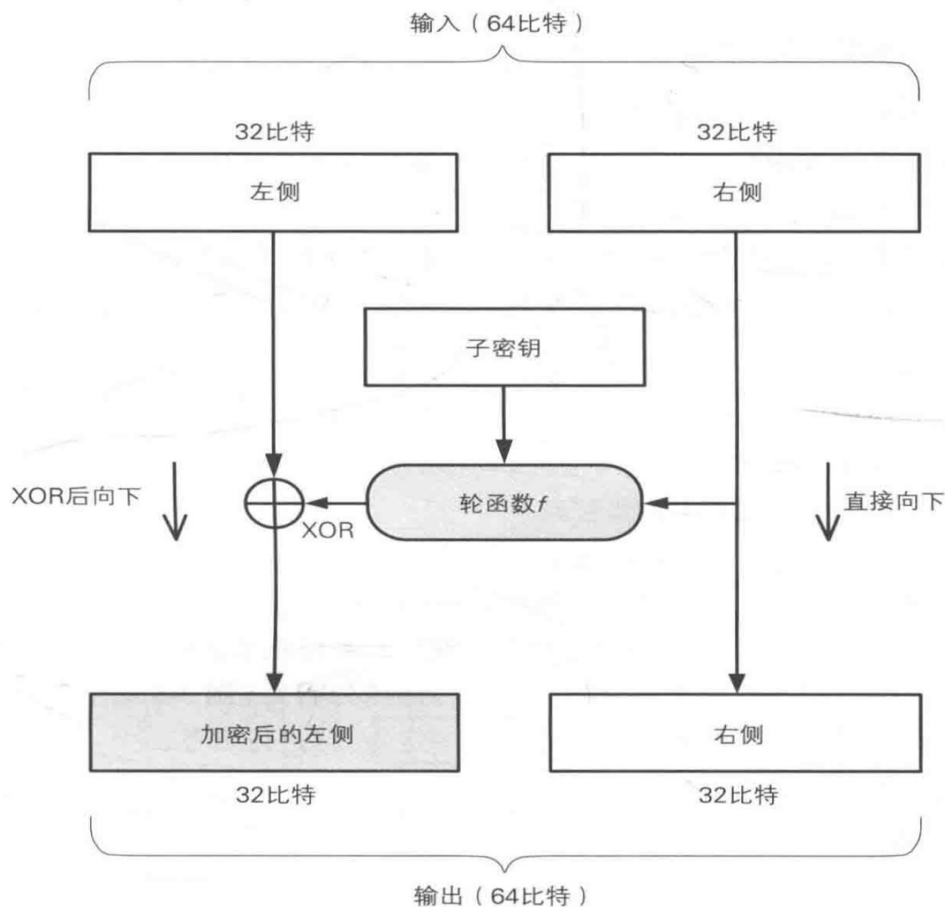
https://www.bilibili.com/video/BV1DS4y1R7hM?spm_id_from=333.788.videopod.sections&vd_source=4c1e046080d396b885383035c58cc631

2.4 DES

- ❑ DES (Data Encryption Standard) 是1977 年美国联邦信息处理标准 (FIPS) 中所采用的一种对称密码(FIPS 46-3)。DES一直以来被美国以及其他国家的政府和银行等广泛使用。
- ❑ 随着计算机的进步, 现在DES 已经能够被暴力破解, 强度大不如前了。1999年的DES Challenge 中只用了22 小时 15 分钟。
- ❑ 由于DES 的密文可以在短时间内被破译, 因此除了用它来解密以前的密文以外, 现在我们应该再使用 DES。
- ❑ DES 是一种将64比特的明文加密成64比特的密文的对称密码算法, 它的密钥长度是 56比特。尽管从规格上来说, DES 的密钥长度是 64 比特, 但由于每隔7比特会设置一个用于错误检查的比特, 因此实质上其密钥长度是 56比特。
- ❑ DES 是以64比特的明文 (比特序列) 为一个单位来进行加密的, 这个64比特的单位称为**分组**。一般来说, 以分组为单位进行处理的密码算法称为**分组密码**(block cipher), DES 就是分组密码的一种。

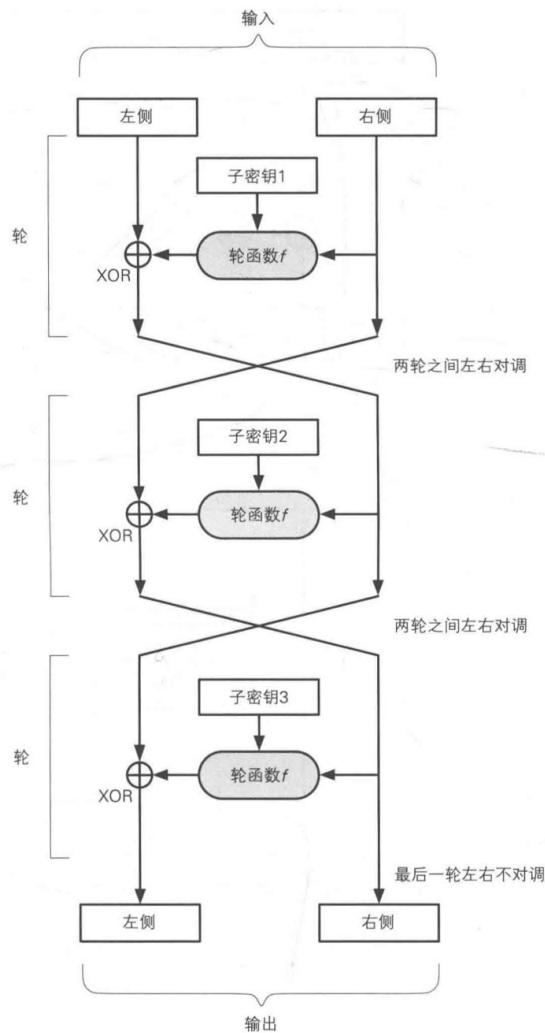
2.4 DES的结构

- DES 的基本结构是由 Horst Feistel 设计的，因此也称为 Feistel 网络(Feistel network)。
- 在Feistel 网络中，加密的各个步骤称为**轮** (round)，整个加密过程就是进行若干次轮的循环。
- DES 是一种16轮循环的Feistel 网络，每轮需要使用一个不同的子密钥。



2.4 DES的结构

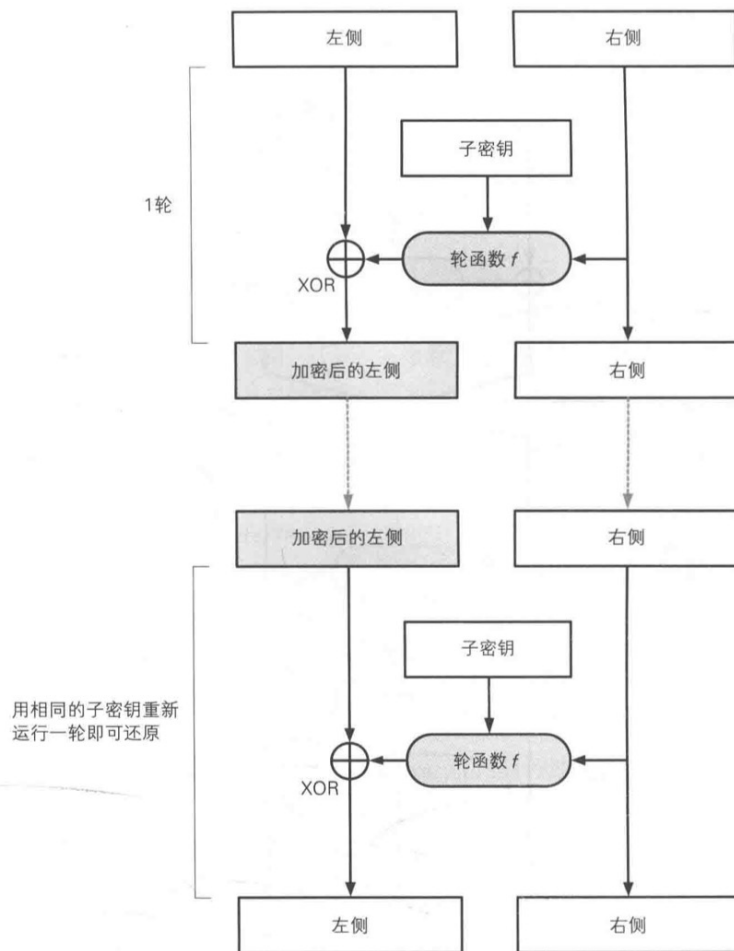
□ 3轮加密2次左右对调



2.4 DES的结构

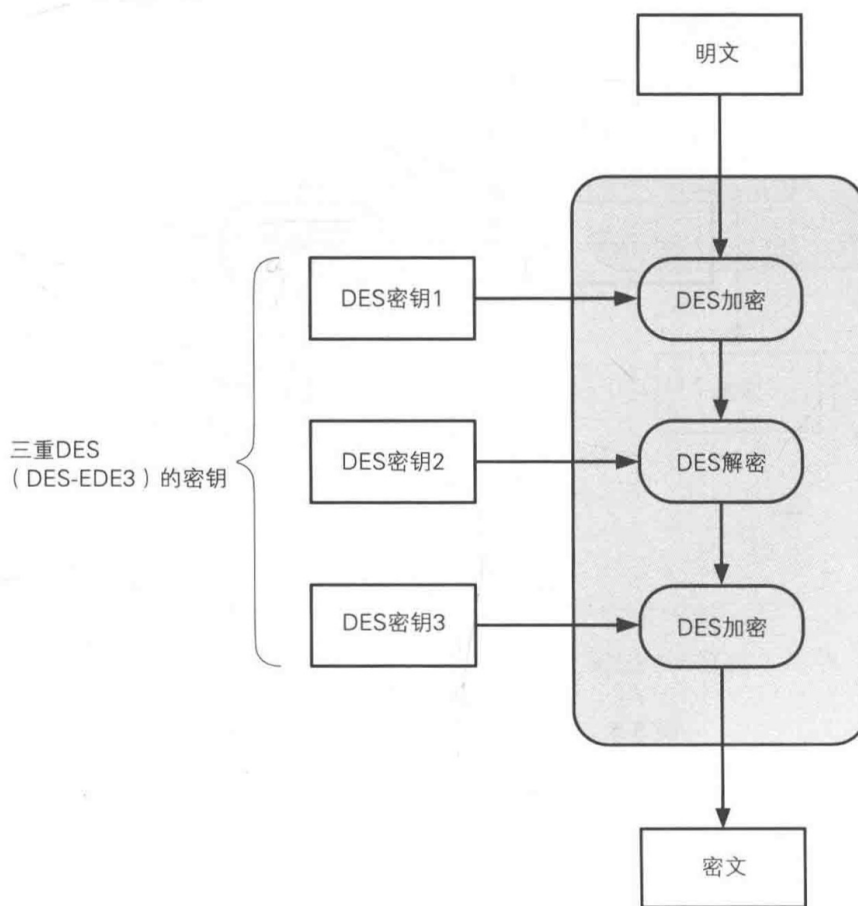
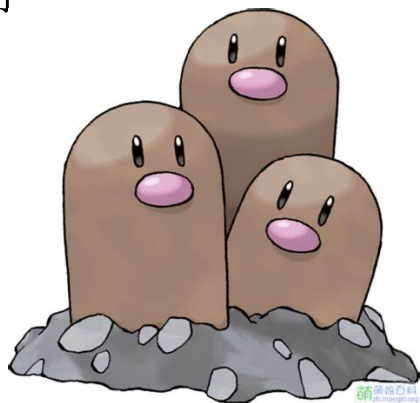
- 解密：根据XOR的性质，用相同的子密钥运行2次Feistel网络就能将数据还原
- 总结
 - Feistel网络的轮数可以任意增加。
 - 加密时无论使用任何函数作为轮函数都可以正确解密。
 - 加密和解密可以使用相同的结构来实现。
 - DES算法已经可以在现实的时间内被暴力破解。

如何增加DES的复杂程度使得难以破解？



2.4 3DES

- ❑ 为了增加DES的强度，将DES重复3次所得到的一种密码算法。
- ❑ 3DES密钥的长度是 $56 \times 3 = 168$ 比特。
- ❑ 如果密钥使用相同，相当于DES。
- ❑ 处理效率不高



2.5 AES

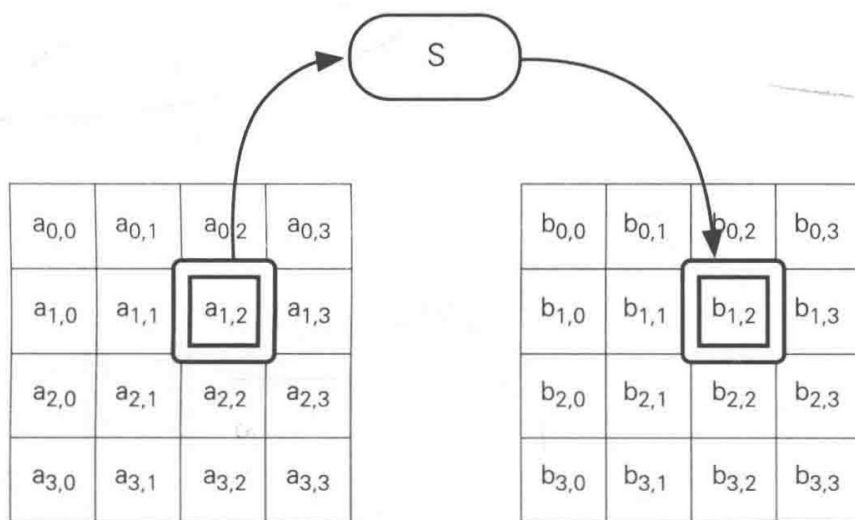
- ❑ AES (Advanced Encryption Standard) 是取代其前任标准(DES)而成为新标准的一种对称密码算法。全世界的企业和密码学家提交了多个对称密码算法作为 AES 的候选，最终在 2000 年从这些候选算法中选出了一名名为 Rijndael 的对称密码算法，并将其确定为了 AES。
- ❑ 参加的条件就是：被选为 AES 的密码算法必须无条件地免费供全世界使用。
- ❑ 参加者还必须提交密码算法的详细规格书、以 ANSI C 和 Java 编写的实现代码以及抗密码破译强度的评估等材料。因此，参加者所提交的密码算法，必须在详细设计和程序代码完全公开的情况下，依然保证较高的强度，这就彻底杜绝了隐蔽式安全性。

2.5 AES （ Rijndael ）

- ❑ Rijndael 是由比利时密码学家 Joan Daemen 和 Vincent Rijmen 设计的分组密码算法，2000 年被选为新一代的标准密码算法AES。今后会有越来越多的密码软件支持这种算法。
- ❑ Rijndael 的分组长度和密钥长度可以分别以 32 比特为单位在 128 比特到 256 比特的范围内进行选择。不过在 AES 的规格中，分组长度固定为 128 比特，密钥长度只有128、192 和 256比特三种。
- ❑ Rijndael 算法也是由多个轮构成的，其中每一轮分为 字节替换（SubBytes）、平移行（ShiftRows）、混合列（MixColumns）和轮密钥加（AddRoundKey）共4个步骤。Rindael没有使用 Feistel 网络，而是使用了 SPN结构。

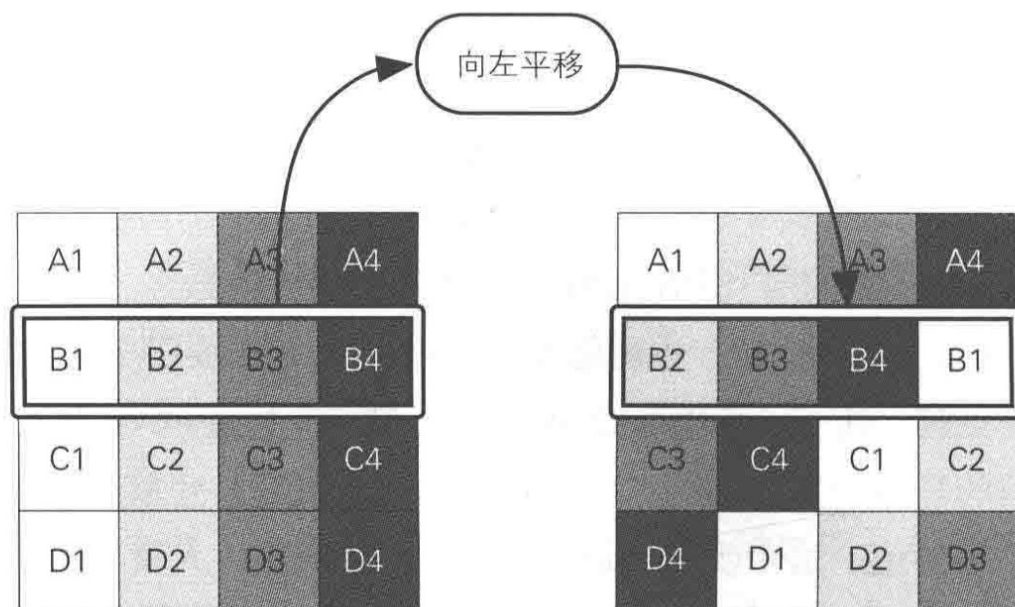
2.5 AES (Rijndael)

- **字节替换:** Rijndael 的输入分组为 128 比特，也就是16字节。首先，需要逐个字节地对 16字节的输入数据进行字节替换处理，是以每个字节的值（0~255 的任意值）为索引，从一张拥有 256个值的替换表(S-Box)中查找出对应值的处理。也就是说，要将一个1字节的值替换成另一个1字节的值。



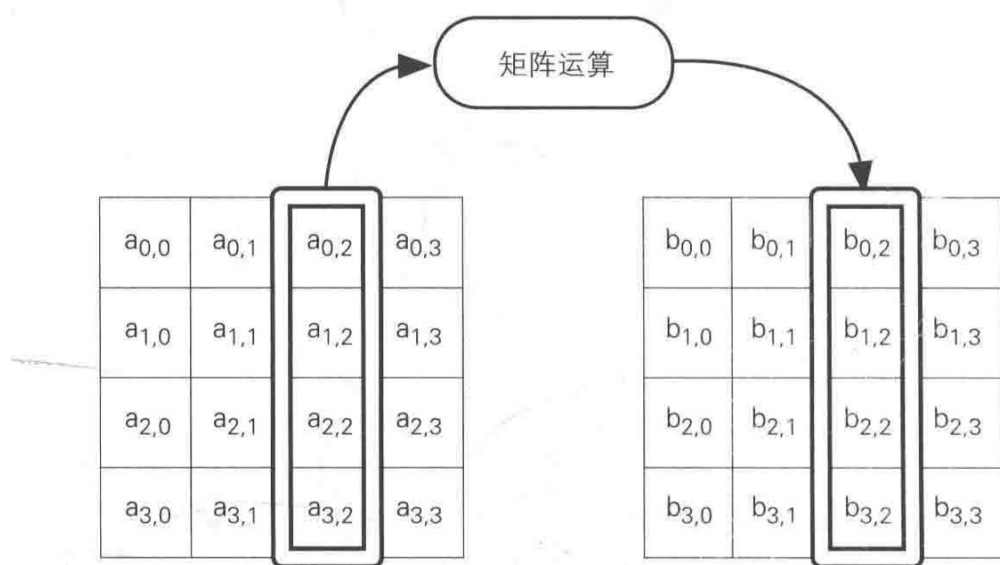
2.5 AES (Rijndael)

- **平移行：** 平移行是指将以4字节为单位的行(row) 按照一定的规则向左平移，且每一行平移的字节数是不同的。



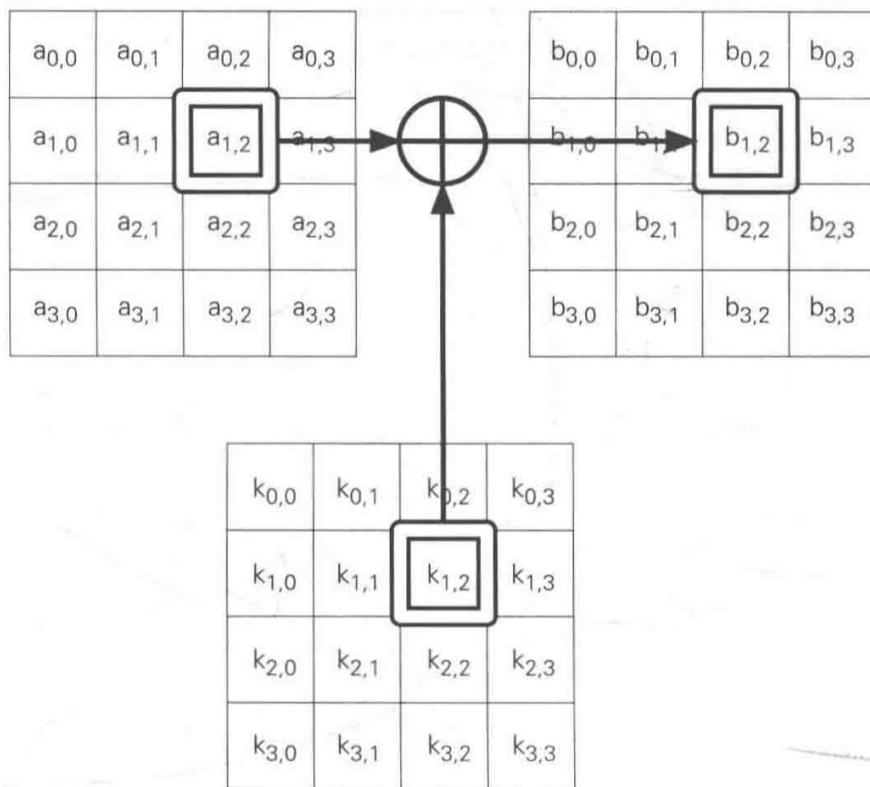
2.5 AES (Rijndael)

- **混合列：**混合列是对一个4字节的值进行比特运算，将其变为另外一个4字节值。



2.5 AES (Rijndael)

- **轮密钥加：** 需要将混合列的输出与轮密钥进行 XOR，即进行轮密钥加处理。实际上，在Rijndael 中需要重复进行 10~14 轮计算。



2.5 AES (Rijndael)

- 输入比特在一轮中都会被加密
- 加密轮数比Feistel网络少 (AES: 10-14; Feistel :16)
- 一轮中的操作可并行处理
- 解密采取相反的步骤
 - 加密：字节替换 -> 平移行 -> 混合列 -> 轮密钥加
 - 解密：轮密钥加 -> 逆混合列 -> 逆平移行 -> 逆字节替换
- 有严格的数学表达与计算



2.6 分组密码的模式

- DES 和 AES 都属于分组密码，它们只能加密固定长度的明文。如果需要加密任意长度的明文，就需要对分组密码进行迭代，而分组密码的迭代方法就称为分组密码的“模式”。
- 密码算法可以分为分组密码和流密码两种。
- **分组密码 (block cipher)** 是每次只能处理特定长度的一块数据的一类密码算法，这里的一块就称为分组 (block)。此外，一个分组的比特数就称为分组长度。
- DES 和 3DES 的分组长度都是 64 比特。这些密码算法一次只能加密 64 比特的明文，并生成 64 比特的密文。

2.6 分组密码的模式

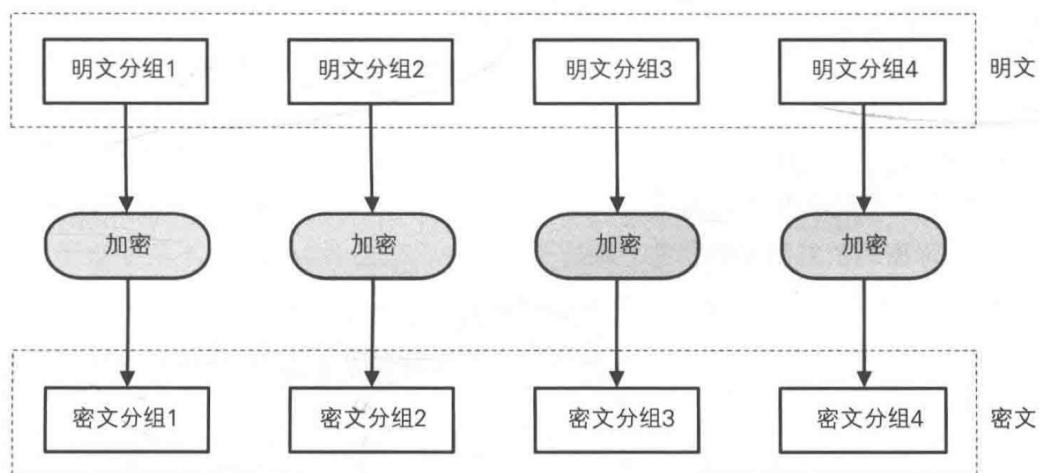
- **流密码(stream cipher)** 是对数据流进行连续处理的一类密码算法，流密码中一般以1比特8比特或 32比特等单位进行加密和解密。
- 分组密码处理完一个分组就结束了，因此不需要通过内部状态来记录加密的进度；相对的，流密码是对一串数据流进行连续处理，因此需要保持内部状态。

思考：最简单的分组方案？

2.6 分组密码的模式

- ❑ **ECB**（Electronic CodeBook）模式：将明文分组加密之后的结果将直接成为密文分组。
- ❑ 使用 ECB 模式加密时，组成“明文分组—密文分组”的对应表，因此 ECB 模式也称为**电子密码本模式**。
- ❑ 当最后一个明文分组的内容小于分组长度时，需要用一此特定的数据进行**填充**（padding）。

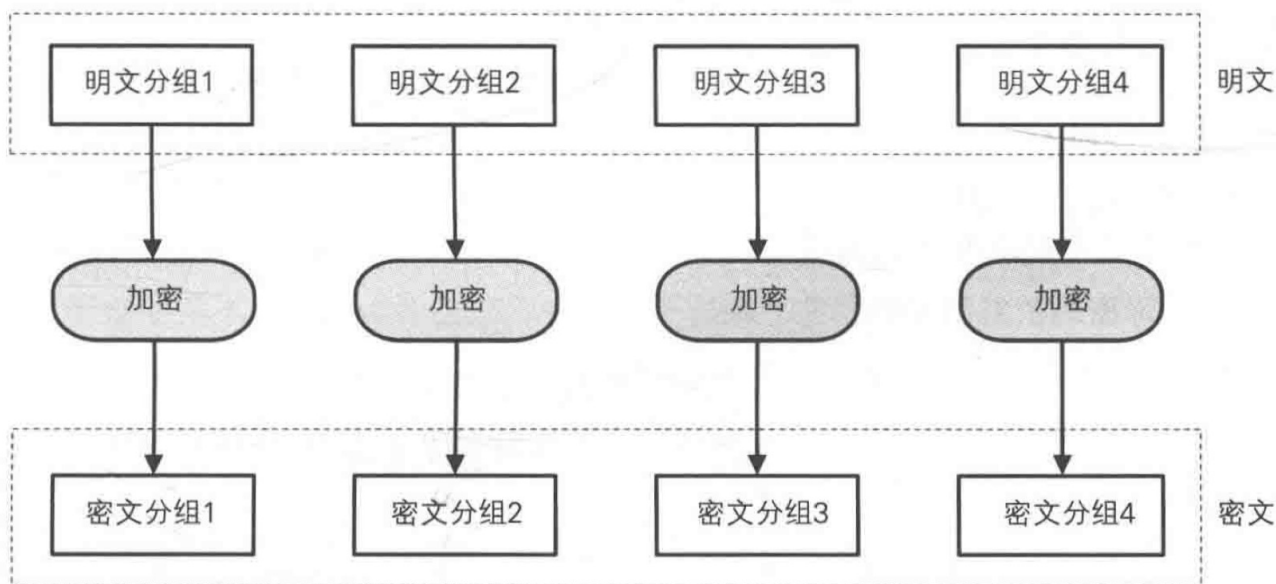
ECB模式的加密



2.6 分组密码的模式

□ 思考：ECB模式是否安全？

ECB模式的加密



2.6 分组密码的模式

- 假如存在主动攻击者，他能够改变密文分组的顺序。当接收者对密文进行解密时，由于密文分组的顺序被改变了，因此相应的明文分组的顺序也会被改变。也就是说，**攻击者无需破译密码就能够操纵明文。**

分组 1 = 付款人的银行账号

分组 2 = 收款人的银行账号

分组 3 = 转账金额

2.6 分组密码的模式

明文分组 1 = 41 2D 35 33 37 34 20 20 20 20 20 20 20 20 20 20 (付款人: A-5374)

明文分组 2 = 42 2D 36 36 37 31 20 20 20 20 20 20 20 20 20 20 (收款人: B-6671)

明文分组 3 = 31 30 30 30 30 30 30 30 30 30 20 20 20 20 20 20 (转账金额: 100000000)

使用ECB模式进行加密:

密文分组 1 = 59 7D DE CC EF EC BA 9B BF 83 99 CF 60 D2 59 B9 (付款人: ????)

密文分组 2 = DF 49 2A 1C 14 8E 18 B6 53 1F 38 BD 5A A9 D7 D7 (收款人: ????)

密文分组 3 = CD AF D5 9E 39 FE FD 6D 64 8B CC CB 52 56 8D 79 (转账金额: ????)

将密文分组1和2的内容对调:

→ 密文分组 1 = DF 49 2A 1C 14 8E 18 B6 53 1F 38 BD 5A A9 D7 D7 (付款人: ????)

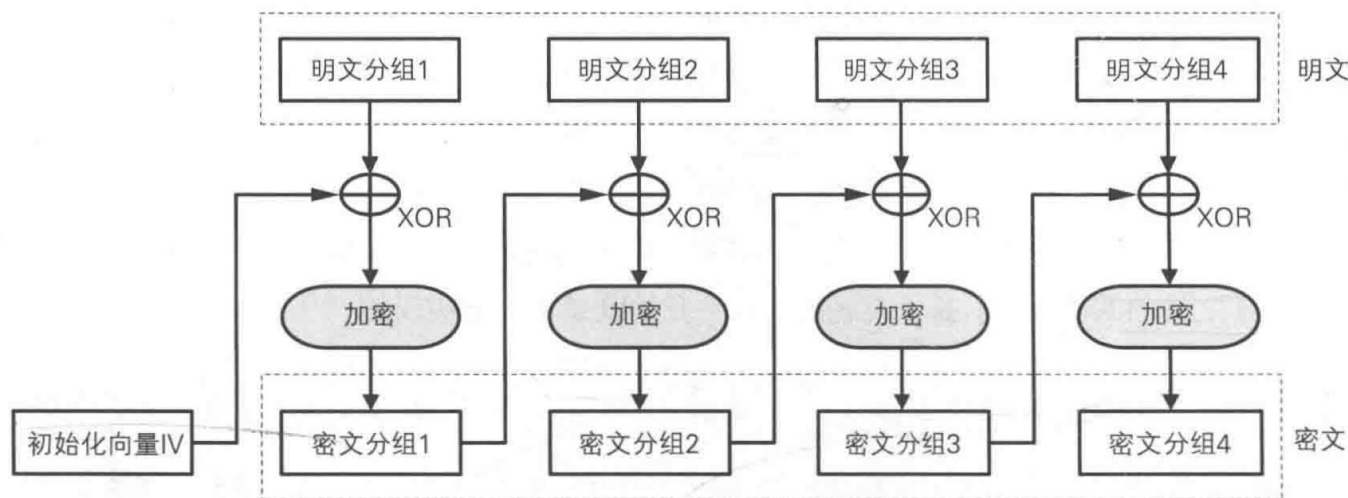
→ 密文分组 2 = 59 7D DE CC EF EC BA 9B BF 83 99 CF 60 D2 59 B9 (收款人: ????)

密文分组 3 = CD AF D5 9E 39 FE FD 6D 64 8B CC CB 52 56 8D 79 (转账金额: ????)

2.6 分组密码的模式

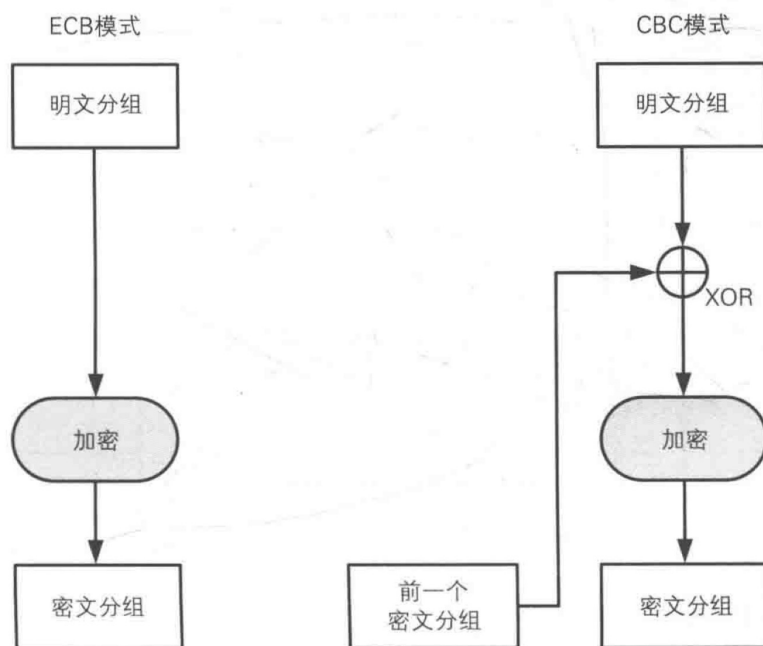
- CBC 模式的全称是 Cipher Block Chaining 模式（密文分组链接模式）。在CBC 模式中，首先将明文分组与前一个密文分组进行 XOR 运算，然后再进行加密。

CBC模式的加密



2.6 分组密码的模式

- ECB 模式只进行了加密，而CBC模式则在加密之前进行了一次 XOR。

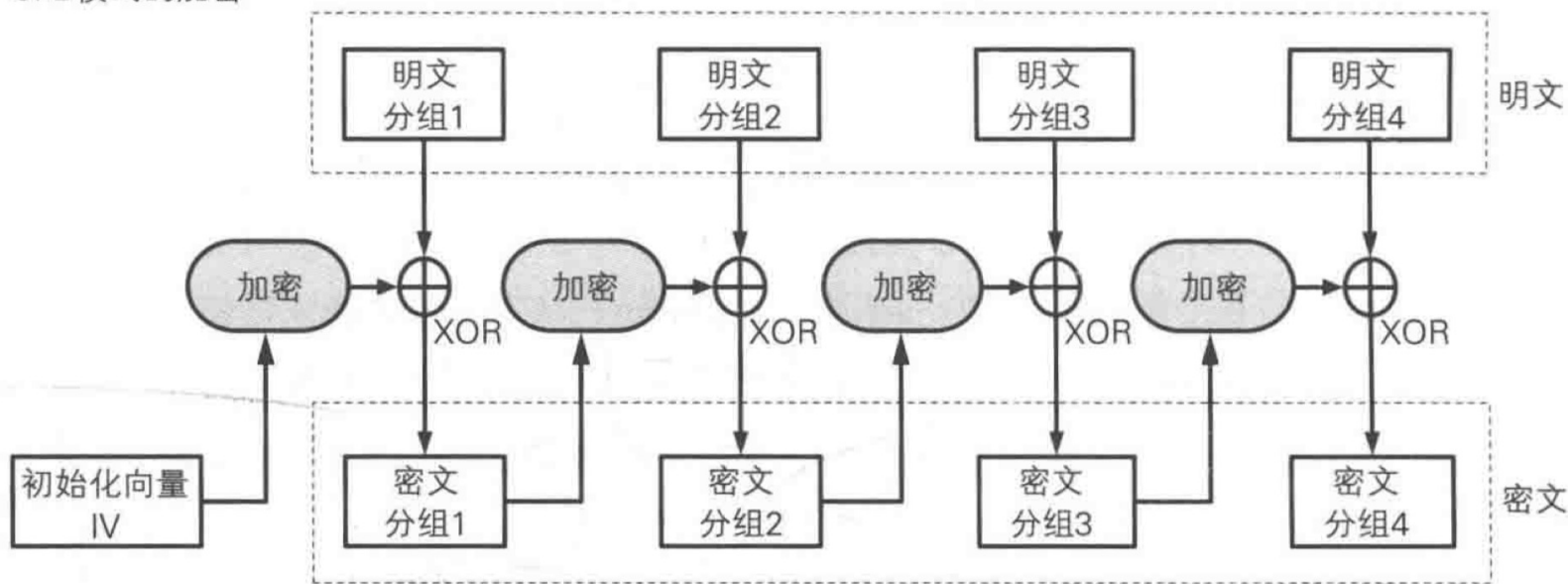


※由于密文分组是相互连接的，因此被称为CBC（密文分组链接）模式

2.6 分组密码的模式

- **CFB 模式**的全称是 Cipher FeedBack 模式（密文反馈模式）。在CFB 模式中，前一个密文分组会被送回到密码算法的输入端。

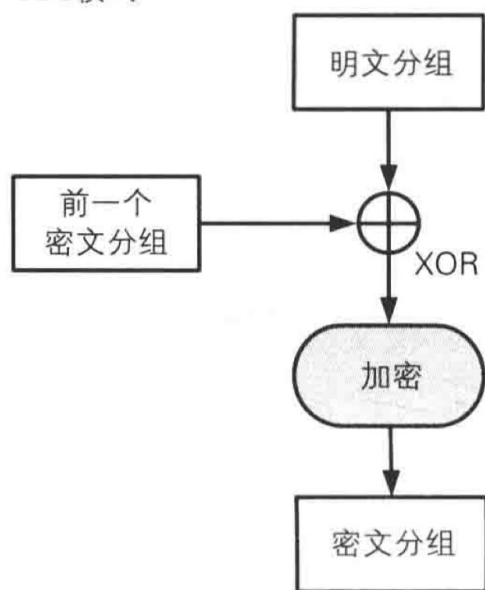
CFB模式的加密



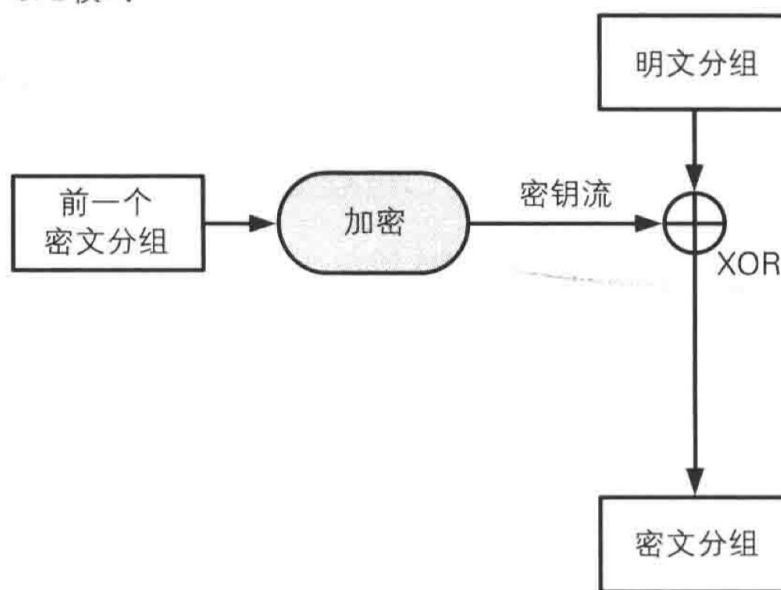
2.6 分组密码的模式

- ❑ CBC 模式中，明文分组都是通过密码算法进行加密的，然而在CFB模式中，明文分组并没有通过密码算法来直接进行加密。
- ❑ 在CFB模式中，明文分组和密文分组之间只有一个 XOR，明文分组和密文分组之间并没有经过“加密”这一步骤。

CBC模式



CFB模式

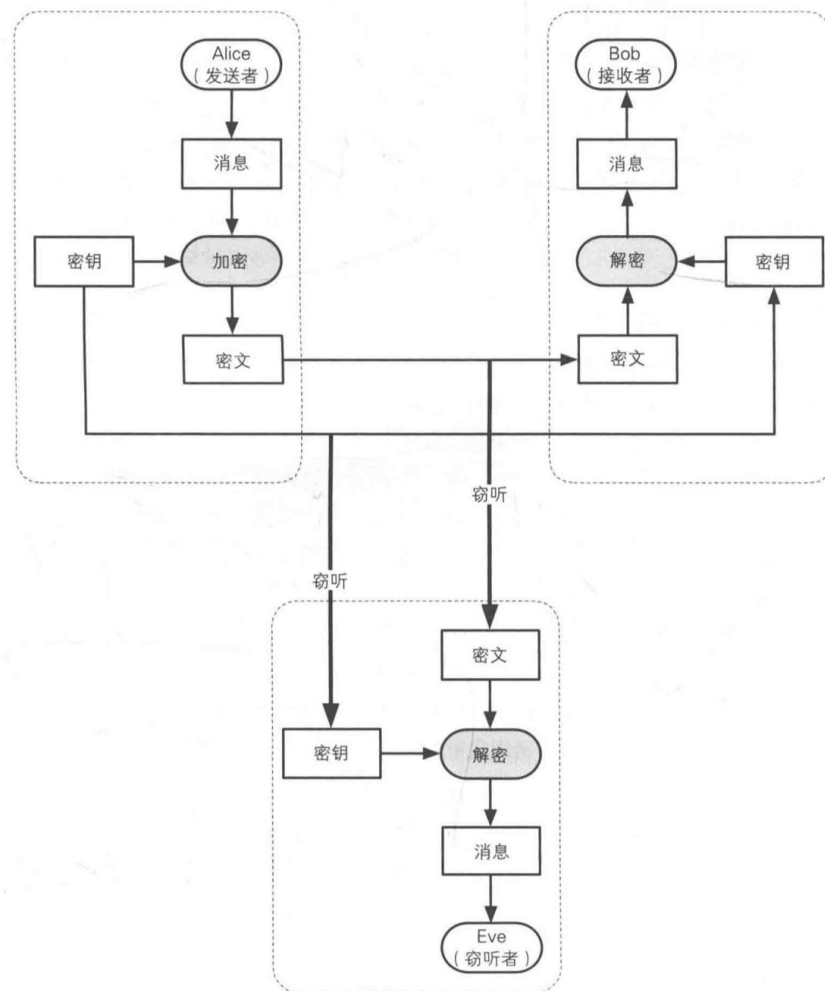


2.6 分组密码的模式

- CFB模式的结构与一次性密码本是非常相似的。
- 在 CFB 模式中，密码算法的输出相当于一次性密码本中的随机比特序列。由于密码算法的输出是通过计算得到的，并不是真正的随机数，因此 CFB 模式不可能像一次性密码本那样具备理论上不可破译的性质。
- CFB 模式中由密码算法所生成的比特序列称为**密钥流**（key stream）。
- 在CFB模式中，明文数据可以被逐比特加密，因此我们可以将 CFB 模式看作是一种**使用分组密码来实现流密码**的方式。

2.7 密钥配送问题

- 如何安全地将密钥分发给需要解密的人？

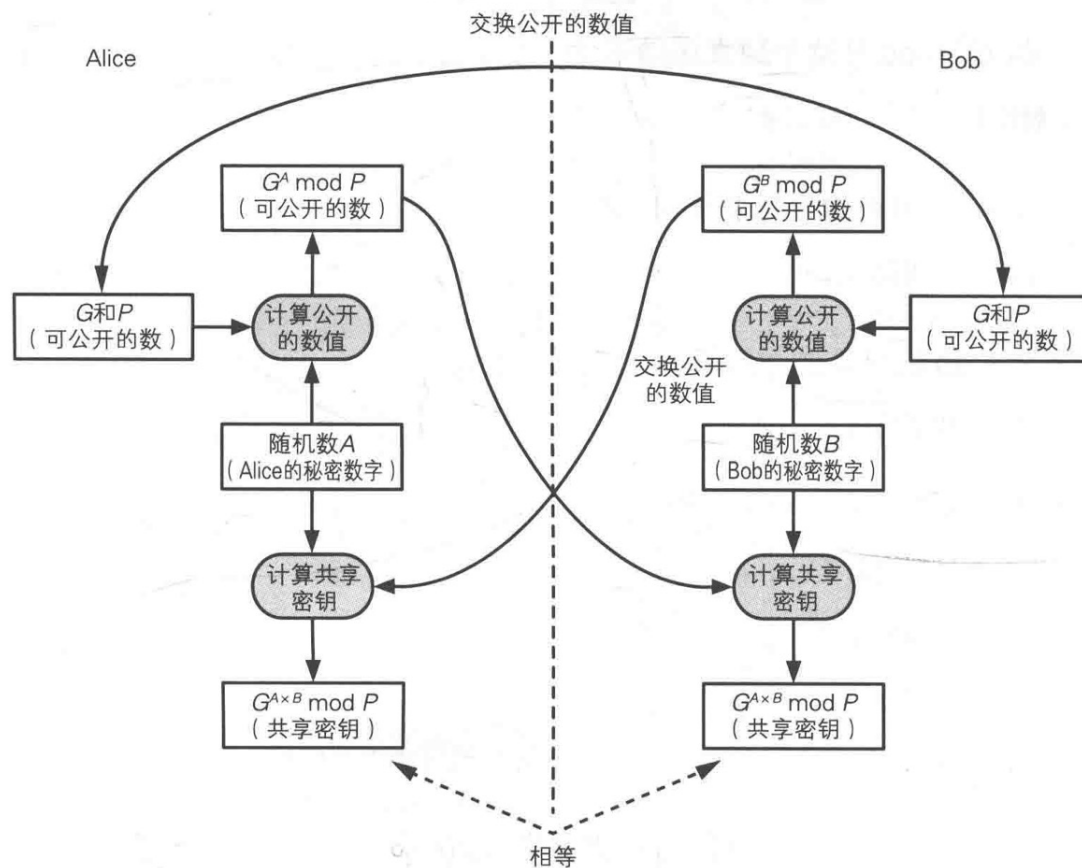


2.7 Diffie-Hellman 密钥交换

- Diffie-Hellman 密钥交换 (Diffie-Hellman key exchange) 是 1976年由 Whitfield Diffie 和Martin Hellman 共同发明的一种算法。使用这种算法，通信双方仅通过交换一些可以公开的信息就能够生成出共享的秘密数字，而这一秘密数字就可以被用作对称密码的密钥。IPsec 中就使用了经过改良的 Diffie-Hellman 密钥交换。
- 虽然这种方法的名字叫“密钥交换”，但实际上双方并没有真正交换密钥，而是通过计算生成出了一个相同的共享密钥。因此，这种方法也称为 Diffie-Hellman 密钥协商(Diffie-Hellman key agreement)。

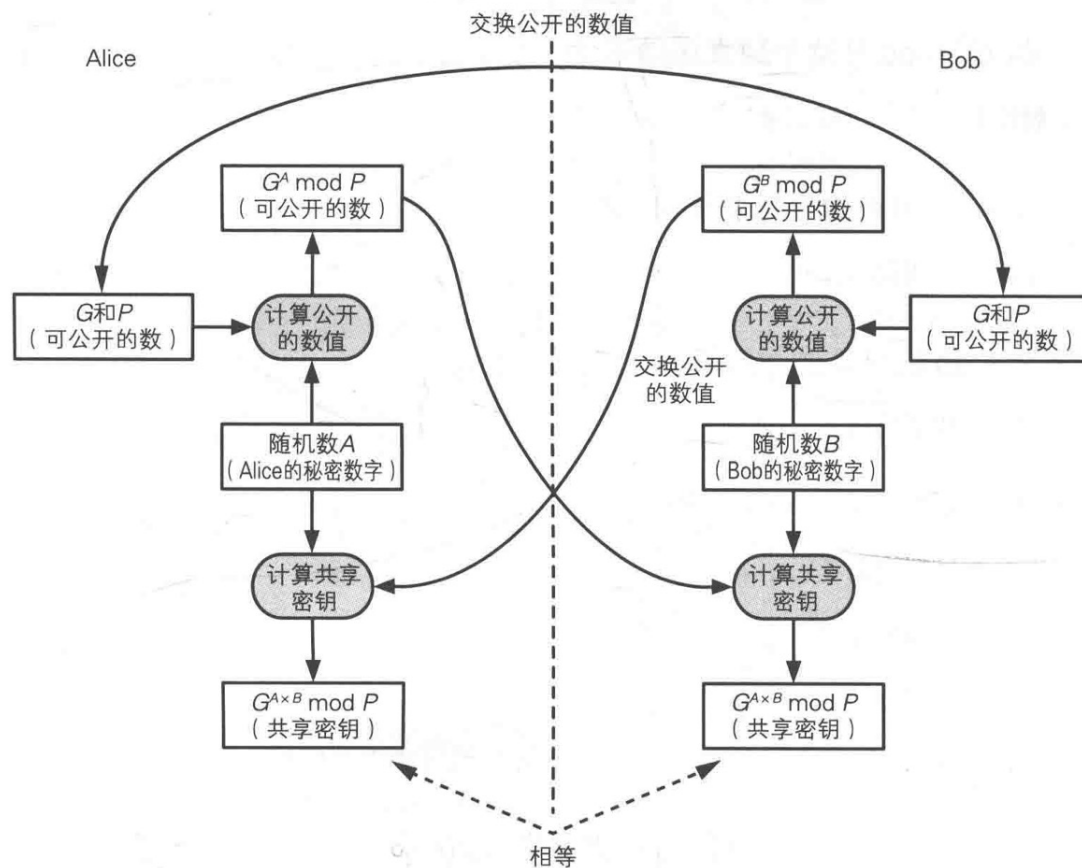
2.7 Diffie-Hellman 密钥交换

- 1. Alice 向Bob 发送两个质数P和G。
- P必须是一个非常大的质数，而G则是一个和P相关的数，称为生成元(generator)。G可以是一个较小的数字。
- P和G 不需要保密，被窃听者获取也没关系。
- P和G可以由 Alice 和Bob 中的任意一方生成。



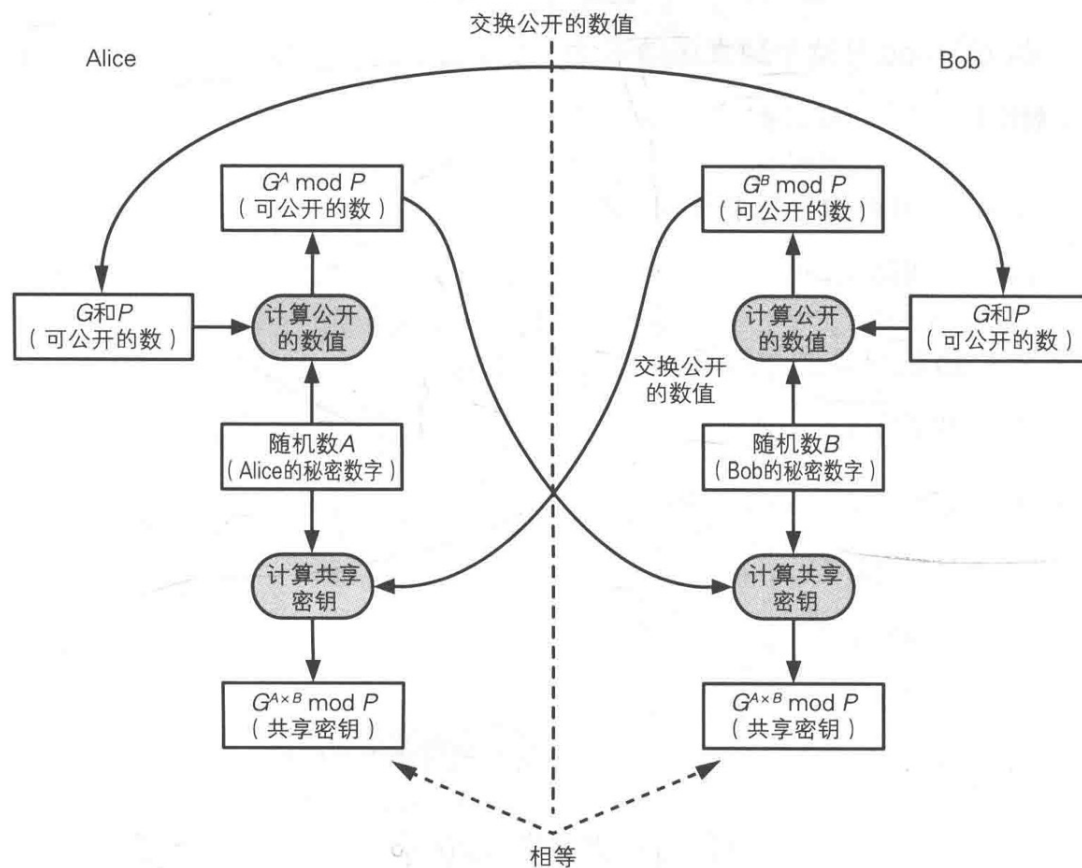
2.7 Diffie-Hellman 密钥交换

- 2. Alice 生成一个随机数 A
 - A 是一个 $1 \sim P-2$ 之间的整数。这个数是一个只有 Alice 知道的秘密数字。
- 3. Bob 生成一个随机数 B
 - B 是一个 $1 \sim P-2$ 之间的整数。这个数是一个只有 Bob 知道的秘密数字。



2.7 Diffie-Hellman 密钥交换

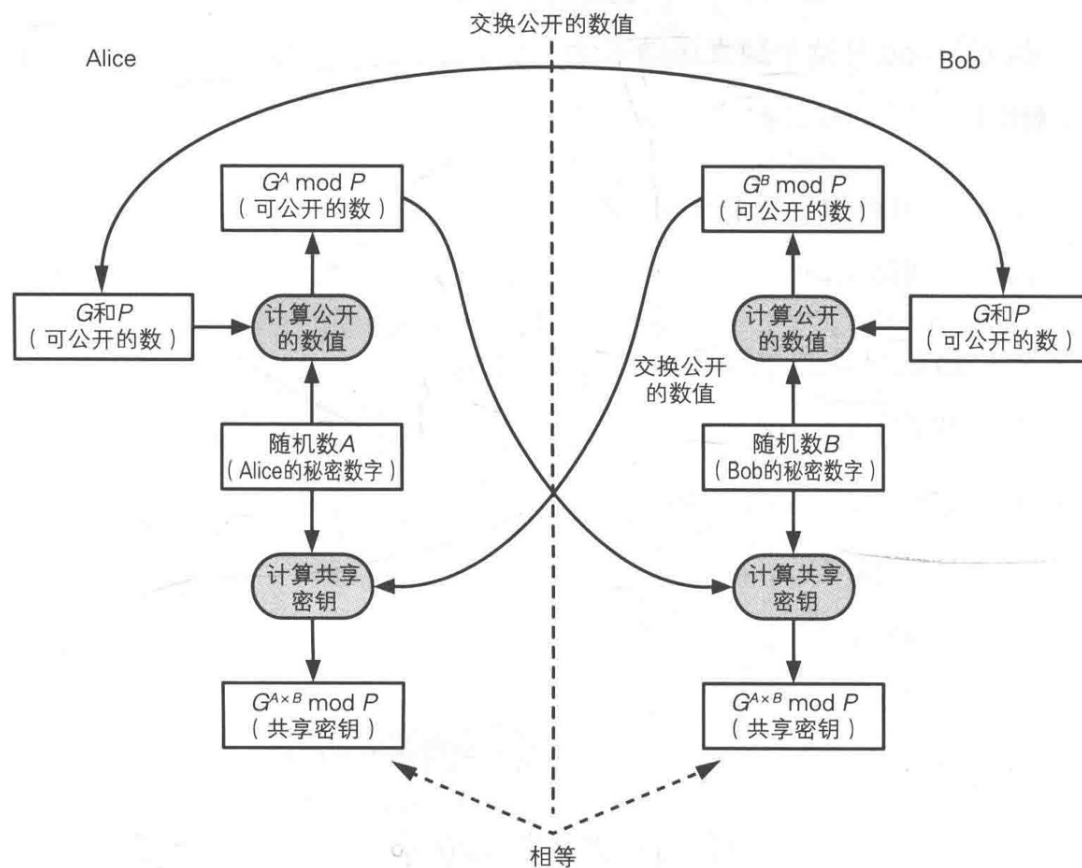
- 4. Alice 将 $G^A \bmod P$ 这个数发送给 Bob
- 被窃听也没关系
- 5. Bob 将 $G^B \bmod P$ 这个数发送给 Alice
- 被窃听也没关系



2.7 Diffie-Hellman 密钥交换

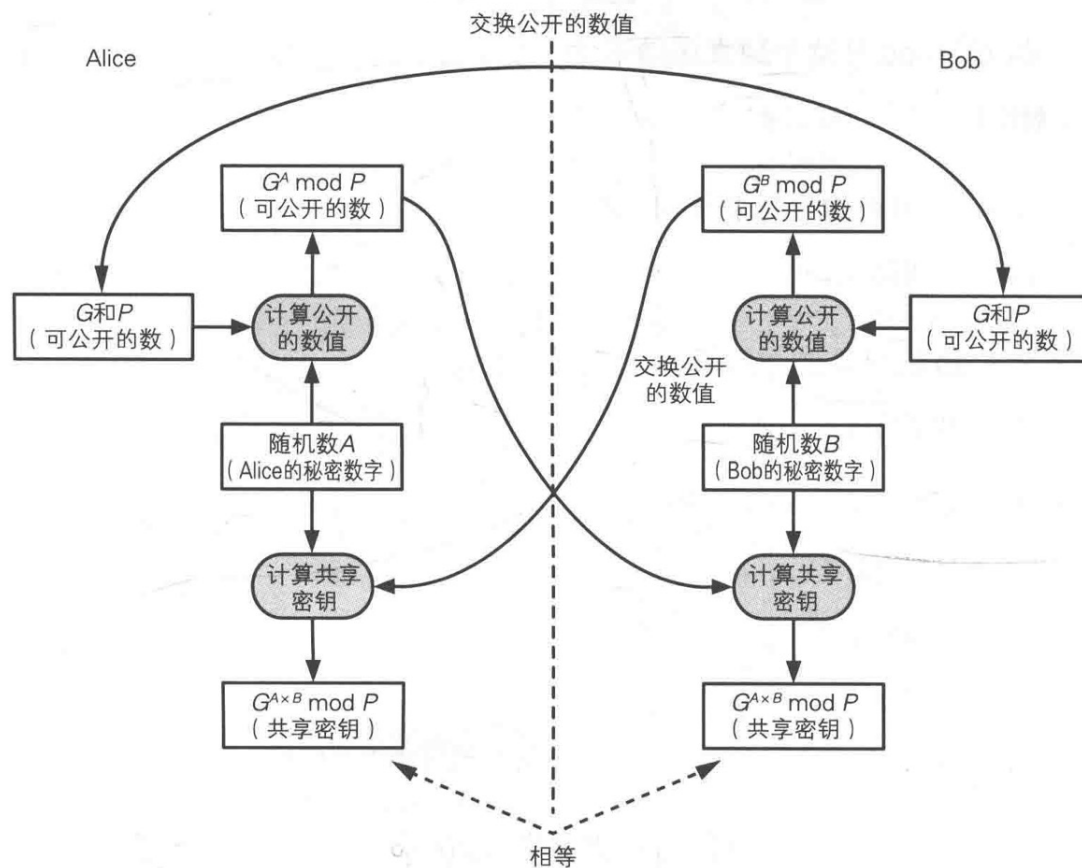
- 6. Alice 用 Bob 发过来的数计算 A 次方并求 $\text{mod } P$
- 7. Bob 用 Alice 发过来的数计算 B 次方并求 $\text{mod } P$

Alice 计算的密钥
= Bob 计算的密钥



2.7 Diffie-Hellman 密钥交换

- 窃听者所知的数据： $P, G, G^A \bmod P, G^B \bmod P$ 。
- 根据以上4个数求解 $G^{(A*B)} \bmod P$ 是非常困难的。
- 有限域的离散对数问题**的复杂度是支撑 Diffie-Hellman 密钥交换算法的基础。



2.7 Diffie-Hellman 密钥交换举例

- 1. Alice 向 Bob 发送两个质数P和G: $P=13, G=2$
 - 2. Alice 生成一个随机数 $A=9$
 - 3. Bob 生成一个随机数 $B=7$
 - 4. Alice 将 $G^A \bmod P$ 这个数发送给 Bob: $2^9 \bmod 13 = 5$
 - 5. Bob 将 $G^B \bmod P$ 这个数发送给 Alice: $2^7 \bmod 13 = 11$
 - 6. Alice 用Bob 发过来的数 11 计算 A 次方并求mod P: 8
 - 7. Bob 用Alice 发过来的数5 计算 B 次方并求 mod P: 8
- 通过上述步骤, Alice 和Bob 就各自计算出了相等的数8 作为共享密钥。



2.8 基于口令的密码

- 基于口令的密码(Password Based Encryption, PBE)是一种根据口令生成密钥并用该密钥进行加密的方法。其中加密和解密使用同一个密钥。

想确保重要消息的机密性。



将消息直接保存在磁盘上的话，可能会被别人看到。



用密钥 (CEK) 对消息进行加密吧。



但是这次又需要确保密钥 (CEK) 的机密性了。



将密钥 (CEK) 直接保存在磁盘上好像很危险。



用另一个密钥 (KEK) 对密钥进行加密 (CEK) 吧。



等等！这次又需要确保密钥 (KEK) 的机密性了。进入死循环了。



既然如此，那就用口令来生成密钥 (KEK) 吧。



但只用口令容易遭到字典攻击。



那么就用口令和盐共同生成密钥 (KEK) 吧。



盐可以和加密后的密钥 (CEK) 一起保存在磁盘上，而密钥 (KEK) 可以直接丢弃。



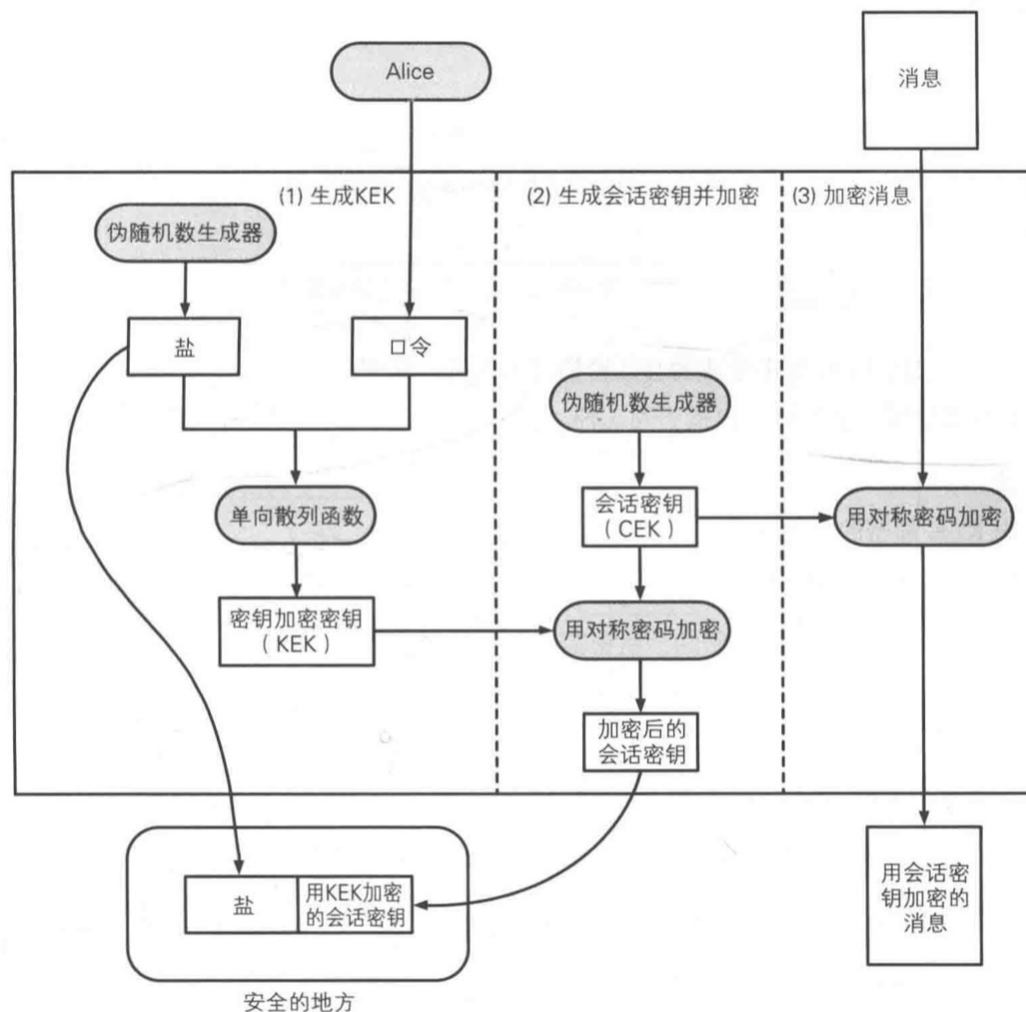
口令就记在自己的脑子里吧。

2.8 基于口令的密码

□ PBE加密包含三步

:

- 生成密钥KEK;
- 生成会话密钥并加密;
- 加密消息。

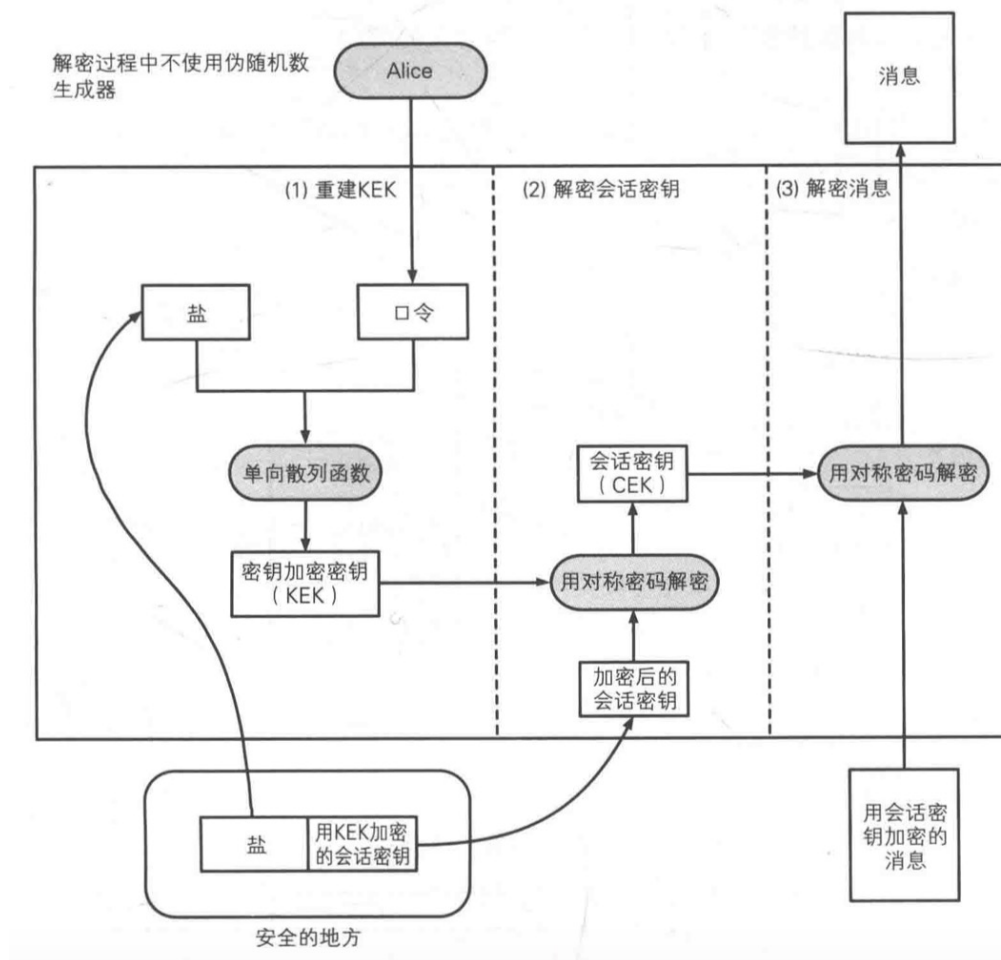


2.8 基于口令的密码

□ PBE解密包含三步

:

- 重建KEK;
- 解密会话密钥;
- 解密消息。



2.8 基于口令的密码

□ 盐的作用：防御字典攻击

➤ 字典攻击是一种事先进行计算并准备好候选密钥列表的方法。

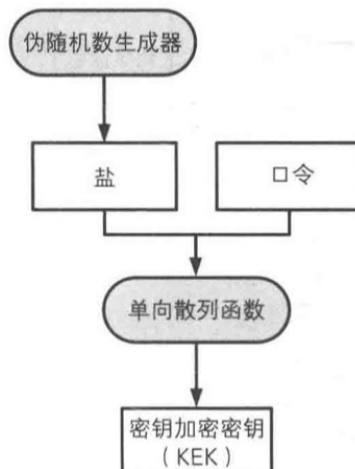
不加盐的情况



攻击者可以事先计算口令所对应的KEK值
(可能进行字典攻击)

口令	对应的KEK值
abc	02 E3 29 13 2A D0
abcde	F5 21 62 FE 72 77
abcxyz	81 75 8E B2 9F 66
hello	3E F3 C7 06 DF B7
pass	18 1C 48 22 E6 EF
...	...

加盐的情况



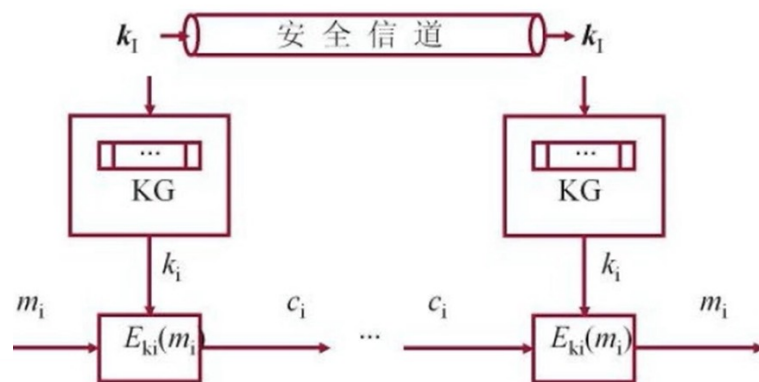
即便口令相同，只要盐不同，KEK值也不同，
因此无法进行字典攻击

盐	口令	对应的KEK值
5B94E7	abc	4D 58 FD 69 87 38
E5AB9D	abc	EB 4D CB A9 C3 A4
F8DC3B	abc	09 70 F0 7D AC 20
C6541B	abc	44 40 32 6F AB 16
F6C109	abc	1F C5 3C 14 DF D8
...	...	...

流密码

- 序列密码，对称密码的一种，实现简单
- 原理：将明文划分成字符(如单个字母),或其编码的基本单元(如0, 1数字), 字符分别与密钥流作用进行加密,解密时以同步产生的同样的密钥流实现;

- 消息流: $m = m_1m_2...m_i$, 其中 $m_i \in M$
- 密文流: $c = c_1c_2...c_i... = E_{k_1}(m_1)E_{k_2}(m_2)...E_{k_i}(m_i)...$, 其中 $c_i \in C$
- 密钥流: $\{k_i\}, i \geq 0$
- 加法流密码: $c_i = E_{k_i}(m_i) = m_i \oplus k_i$



□ 安全性:

- 流密码强度完全依赖于密钥序列的随机性和不可预测性;
- 核心问题是**密钥流生成器**的设计
- 保持收发两端密钥流的精确同步是实现可靠解密的关键技术;

RC4

- ❑ RC4是大名鼎鼎的RSA三人组中的头号人物RonRivest在1987年设计的密钥长度可变的流加密算法。
- ❑ 该算法的速度可以达到DES加密的10倍左右，且具有很高级别的非线性。
- ❑ 一个字节一个字节进行加密
- ❑ 优点：简单灵活，速度快
- ❑ 缺点：安全性低，容易破解

-
- 以一个足够大的数据表为基础，对表进行非线性变化，产生非线性的密钥序列。
 - 基于非线性数据表的序列密钥
 - 置换密码

□ 算法流程

- 初始化S表
- 生成密钥流
- 密钥流与明文异或

初始化S表

- 对S表线性填充，如：

$$S[0]=0, S[1]=1, S[2]=2, \dots, S[255]=255$$

- 用种子密钥填充另外一个K表：

$$K[0], K[1], K[2], \dots, K[255]$$

- $j=0$;

S表一旦初始完成，种子密钥就不再使用了

- For $i=0$ to 255

- $j=j+S[i]+K[i] \bmod 256$

- Swap ($S[i], S[j]$);

举例

0	1	2	3	4	5	6
S[0]	S[1]	S[2]	S[3]	S[4]	S[5]	S[6]

j=0;
For i=0 to 255
 j=j+S[i]+K[i] mod 7
 Swap (S[i],S[j]);

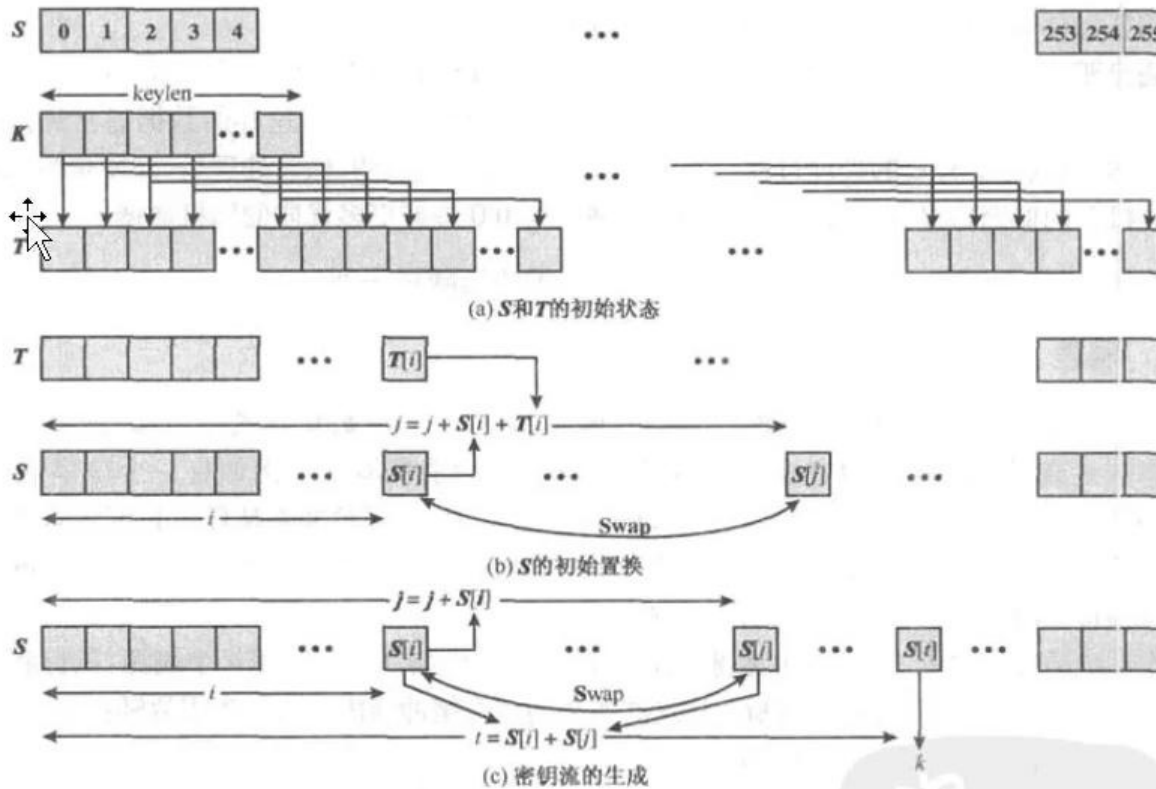
3	4	5	3	4	5	3
K[0]	K[1]	K[2]	K[3]	K[4]	K[5]	K[6]

mod 7

3	0	1	4	5	2	6
S[0]	S[1]	S[2]	S[3]	S[4]	S[5]	S[6]

生成密钥流

- 为每个待加密的字节生成一个用来异或的随机数值
- 循环执行状态转移和输出
 - $i, j = 0;$
 - For $r = 0$ to len // r 为明文字节数
 - $i = (i + 1) \bmod 256$
 - $j = (j + S[i]) \bmod 256$
 - $\text{swap} (S[i], S[j])$ // 状态转移
 - $t := S[(S[i] + S[j]) \bmod 256]$ // 作为 T
 - $K[r] = S[t]$ // 输出



```

i,j=0;
For r=0 to len //r为明文字节数
  i = (i + 1) mod 256
  j = (j + S[i]) mod 256
  swap (S[i, S[j]) //状态转移
  t := S[(S[i] + S[j]) mod 256]
  K[r]=S[t];//输出

```

安全性分析

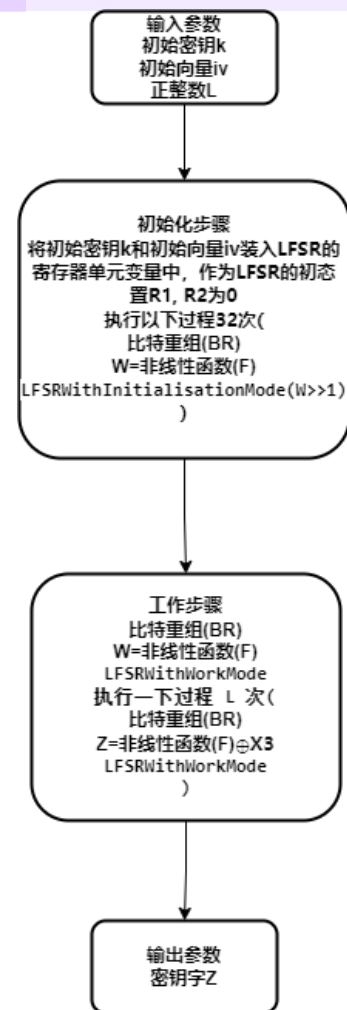
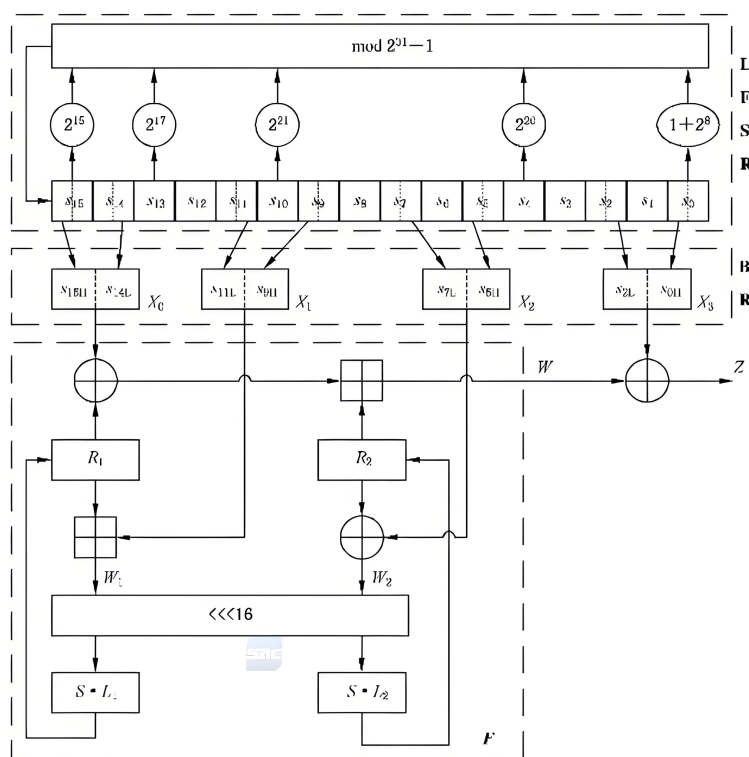
- ❑ RC4衍生出了多个变种算法，如RC4A、RC4X和RC4S等
- ❑ 主要依赖于密钥长度，但是由于RC4算法的密钥流生成过程中存在循环依赖，因此当密钥长度较短时，密钥流的周期较短，容易被暴力破解。
- ❑ RC4算法存在相关密钥攻击、明文攻击等问题

3.3祖冲之密码（ZUC）

- 中国自主研发的流密码算法，被设计用于4G网络及以上的通信系统中
- GB/T 33133.1-2016

流程

- 线性反馈移位寄存器(LFSR)
- 比特重组(BR)
- 非线性函数(F)



线性反馈移位寄存器(LFSR)

- LFSR包括16个31比特寄存器单元S0-S15。
- LFSR的运行模式有两种：初始化模式和工作模式。
- 初始化模式：接收一个31比特的输入u，对S0-S15进行更新

$$(1) v = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0 \bmod (2^{31} - 1);$$

$$(2) s_{16} = (v + u) \bmod (2^{31} - 1);$$

$$(3) \text{如果 } s_{16} = 0, \text{则置 } s_{16} = 2^{31} - 1;$$

$$(4) (s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15}).$$

- 工作模式：无输入，直接更新寄存器单元

$$(1) s_{16} = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0 \bmod (2^{31} - 1);$$

$$(2) \text{如果 } s_{16} = 0, \text{则置 } s_{16} = 2^{31} - 1;$$

$$(3) (s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15}).$$

比特重组(BR)

- 输入为寄存器单元S0-S15，输出为4个32比特字X0, X1, X2, X3

```
BitReconstruction()  
{  
    (1)  $X_0 = s_{15H} \parallel s_{14L}$  ;  
    (2)  $X_1 = s_{11L} \parallel s_{9H}$  ;  
    (3)  $X_2 = s_{7L} \parallel s_{5H}$  ;  
    (4)  $X_3 = s_{2L} \parallel s_{0H}$  .  
}
```

非线性函数(F)

F 包含 2 个 32 比特记忆单元变量 R_1 和 R_2 。

F 的输入为 3 个 32 比特字 X_0, X_1, X_2 , 输出为一个 32 比特字 W 。计算过程如下:

$F(X_0, X_1, X_2)$

{

$$(1) W = (X_0 \oplus R_1) \boxplus R_2;$$

$$(2) W_1 = R_1 \boxplus X_1;$$

$$(3) W_2 = R_2 \oplus X_2;$$

$$(4) R_1 = S[L_1(W_{1L} \parallel W_{2H})];$$

$$(5) R_2 = S[L_2(W_{2L} \parallel W_{1H})]。$$

}

其中 S 为 32 比特的 S 盒变换, S 盒定义见附录 A; L_1 和 L_2 为 32 比特线性变换, 定义如下:

$$L_1(X) = X \oplus (X \lll 2) \oplus (X \lll 10) \oplus (X \lll 18) \oplus (X \lll 24),$$

$$L_2(X) = X \oplus (X \lll 8) \oplus (X \lll 14) \oplus (X \lll 22) \oplus (X \lll 30)。$$

<https://www.cnblogs.com/arduous-termite/articles/17296541.html>



□ 注意：

➤ ZUC-128

- 初始密钥128bit, 初始向量128bit
- 密钥32bit
- 明文长度32bit
- 异或

➤ ZUC-256

- 初始密钥256bit, 初始向量184bit
- 密钥32bit
- 明文长度32bit
- 异或